

ספר פרויקט מערכות אוטונומיות



סמל המוסד: 644450

שם המוסד: מכללת מקיף ה' דרכא אשקלון

שם הסטודנט: גלב שורין

תעודת זהות: 325913580

שם המרצה: משה דזק

הפרויקט: שליטה על זרוע רובוטית באמצעות כפפה.



משרד החינוך

תוכן עיניים

4.....	תקציר
4.....	מטרת העל
5.....	מבוא
5.....	היסטוריה
6.....	סוגים
6.....	מודיפיקציות ו-התקדמות
7.....	מידע נוסף
7.....	דיאגרמת בלוקים לפרויקט
8.....	דיאגרמת מעגלים וסכימות של הזרוע
8.....	דיאגרמת מעגלים וסכימות של הכפפה
9.....	הצעת הבעיה
9.....	הזזת הזרוע באמצעות הכפפה
9.....	הזזת הזרוע באמצעות הטיית היד
10.....	הבעיה עם מד התאוצה
10.....	מה טוב לגבי מד התאוצה?
11.....	בעיות בג'ירוסקופ
11.....	יתרונות הג'ירוסקופ
12.....	Complementary Filter
13.....	חישוב זוויות עבור הפילטר המשלים
14.....	Kalman Filter שיטות חלופיות
14.....	השוואה בין הפילטר המשלים ו-Kalman Filter
15.....	צורת התקשרות ב-MPU6050
16.....	הזזת המנועים על ידי Flex Sensor
17.....	חישוב מיקום ה-flex sensor והמרה קריאה אנלוגית לדיגיטלית
19.....	שליחת הנתונים לזרוע מהכפפה
23.....	הזרוע הרובוטית
23.....	חלקי הזרוע ופעולותיה
24.....	מנועי היד ושליטתם
24.....	מפרט המנוע
25.....	הסבר על האות PWM במנועים

27.....	הסוללה לששת המנועים.....
27.....	בקר השליטה במנועים.....
27.....	תקשורת וקבלת מידע מהכפפה.....
28.....	פונקציות עיקריות בפרויקט.....
28.....	פונקציות חשובות בכפפה.....
31.....	פונקציות חשובות לזרוע.....
32.....	סיכום הפרויקט/רפלקציה.....
32.....	רשימת רכיבים לפרויקט:.....
33.....	ביבליוגרפיה.....
33.....	תמונות של הפרויקט:.....
34.....	קוד פרויקט.....
34.....	קוד הכפפה.....
48.....	קוד הזרוע.....

תקציר

הפרויקט שלי כלל בניית זרוע רובוטית שנשלטת על ידי כפפה בצורה אלחוטית. הזרוע בעלת 6/5 דרגות חופש ונבנתה באמצעות שישה מנועי Servo MG996R שמתופעלים על ידי Servo Module PCA9685 מבית Adafruit שמשתמש בסוללה ו-1 מקבל את הפקודות באמצעות בקר Arduino Uno שמחובר לרכיב התקשורת שלי, רכיב בשם HC05, שעובד באמצעות תקשורת Bluetooth.

הכפפה נבנתה בקפידה והיא משתמשת במיקרו בקר Arduino Nano שמחוברים עליו שלושה flex sensor הבודקים את כיפול האצבעות שלי. בנוסף לזה ישנו גם מד תאוצה וג'ירוסקופ MPU6050 שקולט את תנועת הידיים שלי וכל המידע הזה נשלח לזרוע שתוארה למעלה באמצעות רכיב תקשורת Bluetooth נוסף HC05.

הכפפה "תעתיק" את תנועת היד שלי ותשלח אותה לזרוע הרובוטית ובכך נשלט בזרוע מרחוק. כדי להשיג זאת, השתמשתי בקריאה של אותות אנלוגיים מחיישני הגמישות שלי (flex sensor) מכל אצבע, המרתם לאותיות ושליחת אותיות ה-ASCII לזרוע. בנוסף לזה שילבתי קריאה של המדידות שלי מחיישן ה-MPU6050 שהשתמש ב-Complementary Filter כדי לעבד את נתוני הקלט ובאמצעות שליחה של תווי ASCII לזרוע נוכל לגרום לה לחכות את תנועות היד שלנו.

בסך הכל, הפרויקט שלי הדגים את השימוש בחיישנים כמו MPU6050 ו-Flex Sensor, מיקרו בקרים ובקרים ביניהם Arduino Uno/Nano ושליחת נתונים בצורה אלחוטית באמצעות HC05. הפרויקט הראה גם את החשיבות שיש בשימוש של אלגוריתמים מתאימים והתקני שלט כדי להשיג תוצאות אמינות. בעתיד אפשר לשפר את הפרויקט ולהרחיב אופקים אתו למשל כמו שניית שלט לשלוט בו או שילוב עם טכנולוגיות כמו למידת מוכנה וראיית מחשב לקליטה של מיקום היד שלי.

מטרת העל

מטרת העל של הפרויקט היא ליצור דרך אלטרנטיבית לשליטה בזרוע רובוטית, בניגוד לקונבנציות הידועות הכוללים שלט, זרוע מיניאטורית שמדמה את הזרוע המקורית ועוד.

הפרויקט כולל כפפה וזרוע כך שהכפפה שולחת את הנתונים שהיא קוראת ומעבדת לזרוע והזרוע מגיבה בהתאם וזזה למקום שהכפפה הוראת לה.

בניית הפרויקט כללה תכנון ובחירה קפידה באיזה רכיבים הכי נוח להשתמש בכפפה שלי והם ה-MPU6050 וה-Flex Sensor גם בגלל היותם קטנים וגם בגלל אמינותם, ובזרוע מנועי Servo מסוג MG996R בגלל עוצמתם ודיוקם.

והכי חשוב בחירת רכיב התקשורת ביניהם ה-HC05 רכיב שעובד באמצעות Bluetooth והוא מושלם לעבודה גם בגלל הפשטות בשימוש שלו וגם בגלל היותם קטן ונוח בכפפה.

מבוא

זרוע מכנית היא מכונה שמחכה את פעולות היד האנושית. זרועות מכניות מורכבות ממספר קורות המחוברות על ידי צירים המופעלים על ידי אקטואטורים (מפעילים). קצה אחד של הזרוע מחובר לבסיס יציב בעוד שבשני יש כלי. הם יכולים להישלט על ידי בני אדם ישירות או ממרחק. זרוע מכנית הנשלטת על ידי מחשב נקראת זרוע רובוטית. עם זאת, זרוע רובוטית היא רק אחד מיני סוגים רבים של זרועות מכניות שונות.

זרועות מכניות יכולות להיות פשוטות כמו פינצטה או מורכבות כמו זרועות תותבות. במילים אחרות, אם מנגנון יכול לתפוס חפץ, להחזיק חפץ ולהעביר חפץ בדיוק כמו זרוע אנושית, ניתן לסווג אותו כזרוע מכנית.

זרוע רובוטית היא סוג של זרוע מכנית, סדרך כלל מתוכנתת, אם תכונות דומות ליד אנושית; הזרוע עשויה להיות הסכום הכולל של המנגנון או עשויה להיות חלק מרובוט מורכב יותר. הקישורים של מניפולטור כזה מחוברים על ידי מפרקים המאפשרים תנועה סיבובית (כגון ברובוט מפרקי) או תזוזה טרנסציונלית (לינארית). החוליות של המניפולטור יכולות להיחשב כיוצרות שרשרת קינמטית. הקצה של השרשרת הקינמטית של המניפולטור נקרא אפקטור הקצה והוא מקביל ליד האדם. עם זאת, המונח "יד רובוטית" כמילה נרדפת לזרוע הרובוטית אסור לעתים קרובות.

ההתקדמות האחרונה הובילה לשיפורים עתידיים בתחום הרפואי עם תותבות ועם הזרוע המכנית בכלל. כאשר מהנדסי מכונות בונים זרועות מכניות מורכבות, המטרה היא שהזרוע תבצע משימה שזרועות אנושיות רגילות אינן יכולות להשלים.

היסטוריה

- **זרועות רובוטיות**, שחזה לראשונה על ידי ויליאם פולארד בסוף שנות ה-30, ראו התקדמות משמעותית עם ה-Unimate של ג'ורג' דבול ב-1961, ששימשו למשימות מסוכנות בתעשיות. החידושים נמשכו עם תרומות מהאקדמיה, כמו זרוע סטנפורד של ויקטור שיינמן והזרוע המיומנת של מרווין מינסקי, והעצימו את היכולות שלהם ליישומים שונים.

- **הזרוע המכנית** הראשונה לייצור רכב הוצגה בשנת 1962 במפעל של ג'נרל מוטורס, וחוללה מהפכה במשימות כמו ריתוך והסרת יציקה. עד 1979 פיתחה נחי רובוט מונע ממונע לריתוך נקודתי, חיוני להרכבת מכונות. בשנת 2012, האוניברסיטה הלאומית של סינגפור קידמה את הטכנולוגיה הזו עוד יותר עם זרוע מכנית המסוגלת להרים פי 80 ממשקלה ולהימשך עד פי חמישה מאורכה, מה שהועיל לייצור המכונות באופן משמעותי.

- **זרועות כירורגיות** הופיעו לראשונה בשנת 1985 עם ביופסיה נוירוכירורגית. ה-FDA אישר את מערכת ה-AESOP (Automated Educational Substitute Operator) בשנת 1990. בשנת 2000, הפכה המערכת הכירורגית של דה וינצ'י למערכת הניתוחים הרובוטיים הראשונה שאושרה על ידי ה-FDA, מה שסימן התקדמות משמעותית בניתוחים בעזרת רובוטים.

סוגים

- **רובוט קרטזי/רובוט גאנטרי** – משמש בעיקר לעבודות מערכת אחסון ושליטה אוטומטית
- **Cobot** – ל-Cobot מגוון רחב של שימושים בעיקר ב- יישום מסחרי, מחקר רובוטי, ניפוק, טיפול בחומרים, הרכבה, גימור, בדיקת איכות.
- **רובוט גלילי / Cylindrical robot** – משמש לפעולות הרכבה, טיפול במכונות, ריתוך נקודתי וטיפול במכונות יציקה. זהו רובוט שציריו יוצרים מערכת קואורדינטות גלילית.
- **רובוט כדורי / רובוט קוטבי** – משמש לטיפול במכונות, ריתוך נקודתי, יציקה, מכונות ליטוש, ריתוך גז וריתוך קשת. זהו רובוט שציריו יוצרים מערכת קואורדינטות קוטבית.
- **רובוט SCARA** – משמש לעבודות איסוף ומקום, יישום איטום, פעולות הרכבה וטיפול מכונות. רובוט זה כולל שני מפרקים סיבוביים מקבילים כדי לספק תאימות במטוס.
- **רובוט מפרקי** – משמש לפעולות הרכבה, יציקה, מכונות ליטוש, ריתוך גז, ריתוך קשת וצביעה בהתזה. מדובר ברובוט שלזרועו יש לפחות שלושה מפרקים סיבוביים.
- **רובוט מקביל** – שימוש אחד הוא פלטפורמה ניידת המטפלת בסימולטורים של טיסה בתא הטייס. זהו רובוט שלזרועותיו יש מפרקים מנסרים או סיבוביים במקביל.
- **רובוט אנתרופומורפי** – הוא מעוצב בצורה שמזכירה יד אנושית, כלומר עם אצבעות ואגודלים עצמאיים.

מודיפיקציות ו-התקדמות

- רקמת שריר לזרועות מכניות.
- חיישן זרועות מכניות.
- למידת מכונה וראיית מחשב.

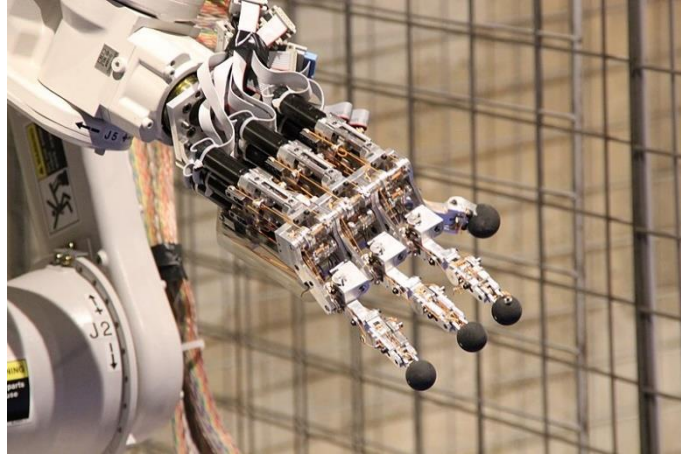
הודות לחיישנים ותכונות עיצוב אחרות כגון חומרים קלים וקצוות מעוגלים, Cobots מסוגלים ליצור אינטראקציה ישירה ובטוחה עם בני אדם.



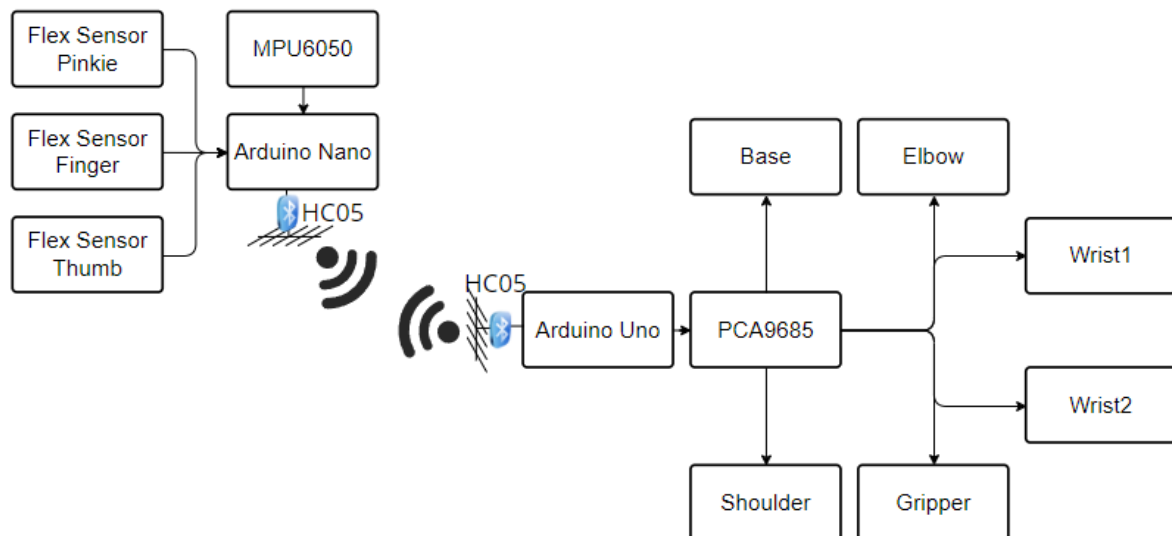
מידע נוסף

- יד רובוטית – ניתן לעצב את האפקטור הקצה, או היד הרובוטית, לביצוע כל משימה רצויה כגון ריתוך, אחיזה, ספינינג וכו', בהתאם ליישום. לדוגמה, זרועות רובוטים בפסי ייצור לרכב מבצעות מגוון משימות כגון ריתוך וסיבוב חלקים והצבה במהלך ההרכבה. בנסיבות מסוימות, רצוי חיקוי קרוב של היד האנושית, כמו ברובוטים שנועדו לבצע פירוק וסילוק פצצות.

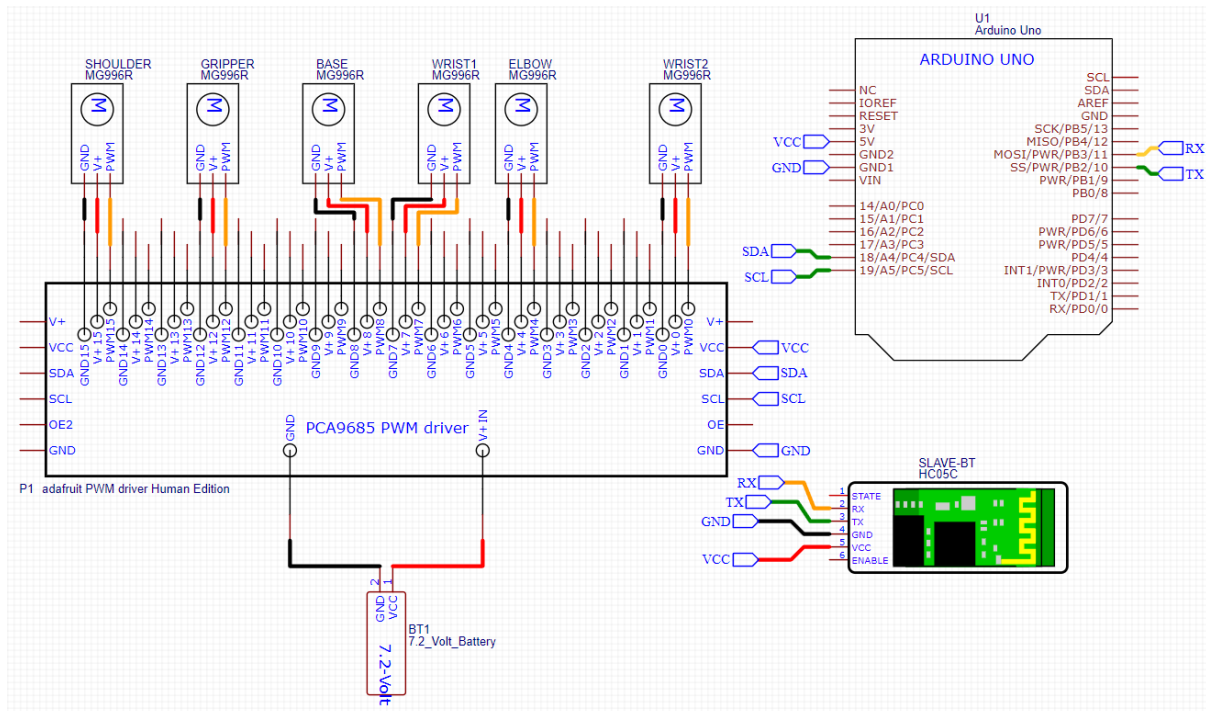
יד רובוטית בדמוי יד של בן אדם.



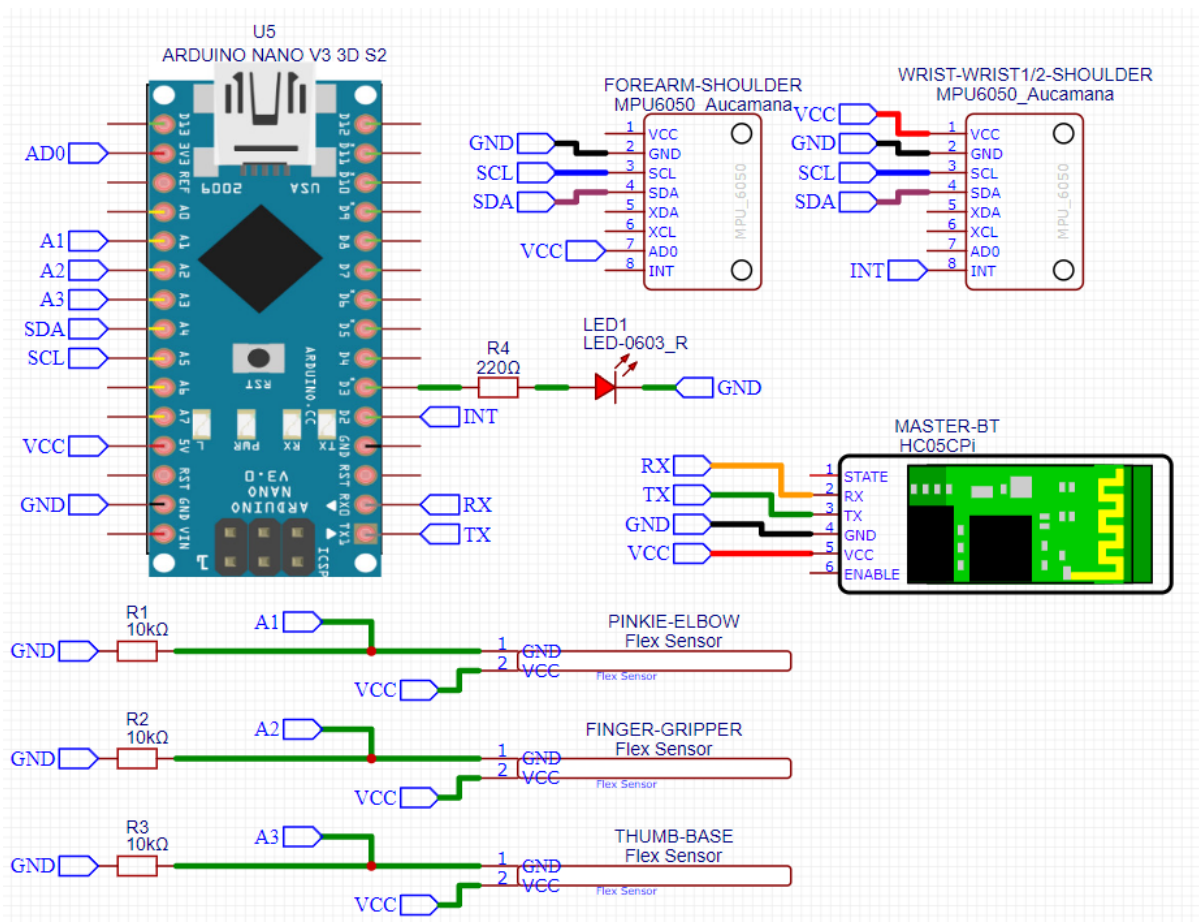
דיאגרמת בלוקים לפרויקט



דיאגרמת מעגלים וסכימות של הדרוע



דיאגרמת מעגלים וסכימות של הכפפה



הצעת הבעיה

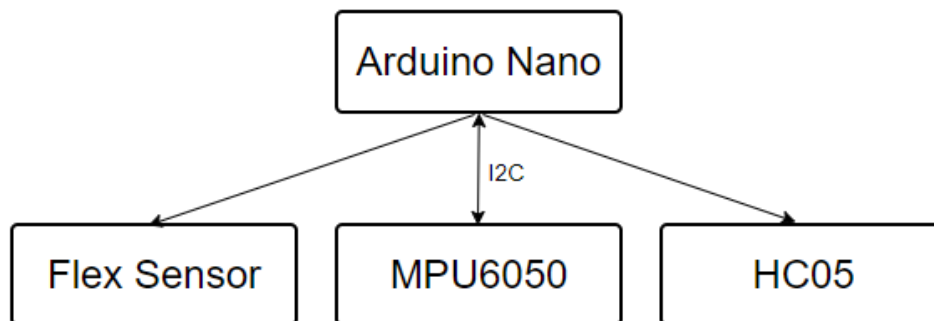
הפרויקט נותן מענה לאנשים שצריכים כלי שימש לעבודות מיון, למידה ועוד.

המטרה העיקרית של הפרויקט היא לספק זרוע רובוטית שיהיה ניתן לשלוט בה על ידי כפפה בתווך רחוק לשלל משימות שונות כמו הרמת דברים, סידור חפצים ועוד.

הפרויקט בה לפתור בעיות נגישות וכדומה ולהוות גם בתור יד רובוטית שכל אחד יכול להרשות לעצמו בבית. אני לוקח רעיונות ועיצובים של זרועות קיימות ויוצר פתרון שהיא זרוע במחיר סביר יותר.

הזזת הזרוע באמצעות הכפפה

אלו החלקים הנדרשים בשביל להזיז את הזרוע עם הכפפה:



הזזת הזרוע באמצעות הטיית היד

פעולת הזזת הזרוע באמצעות הטיית היד תעשה באמצעות שימוש בחיישן ה-MPU6050.

חיישן ה-MPU הוא מכשיר מעקב התנועה היחיד בעולם עם שישה צירים המיועד לדרישות הספק נמוכות, עלות נמוכה וביצועים גבוהים של סמארטפונים, טאבלטים וחיישנים לבישים.

החיישן כולל בתוכו מד תאוצה וג'ירוסקופ ומד טמפרטורה שבוא לא נעשה שימוש בפרויקט זה.

ג'ירוסקופ – גירוסקופ או ג'ירוסקופ ועל פי האקדמיה ללשון העברית סביבון, מכונה לעיתים בקיצור ג'ירור) באנגלית, Gyroscope: מיוונית "גירוס"="עיגול, סיבוב" ו"סקופוס"="ראייה" השם הומצא על ידי הפיזיקאי הצרפתי לאון פוקו ב-1852 (הוא מכשיר מדעי המשמש למדידה או שמירה של יציבות, תוך התבססות על עקרונות שימור התנע הזוויתי. בפיזיקה, שם זה ידוע גם כ"אינרציה גירוסקופית". אחד השימושים הנפוצים של מכשיר זה הוא מדידת זוויות הגוף ביחס למערכת צירים של כדור הארץ.

ב-Arduino IDE קיימת ספרייה אשר יודעת לטפל בקריאות ה-MPU ובעזרת פילטר הנקרא Complementary Filter ניתן להמיר את הקריאות הללו לזוויות הטייה של היד, ובכך למסור לזרוע את הזוויות הנ"ל ולגרום לה לזוז.

הג'ירוסקופ של ה MPU שונה מהג'ירוסקופ הידוע, בכך שרכיב הג'ירוסקופ של MPU6050 מודד מהירות זוויתית או תנועה סיבובית סביב שלושה צירים: X, Y ו-Z. הוא יכול לזהות את קצב השינוי של הסיבוב במעלות לשנייה ($s/^{\circ}$). הג'ירוסקופ מספק מידע על הכיוון והתנועה של אובייקט במרחב תלת ממדי.

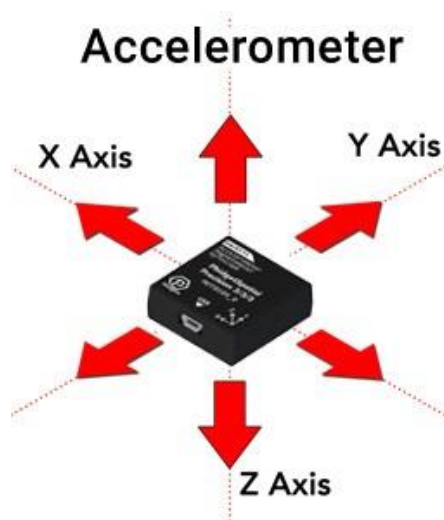
הבעיה עם מד התאוצה

למד התאוצה הקיים ב MPU6050, כמו לכל חיישן, יש מגבלות מסוימות ובעיות פוטנציאליות שהמשתמשים צריכים להיות מודעים להן. להלן כמה אתגרים נפוצים הקשורים לרכיב מד התאוצה של ה-MPU6050:

- (1) **היסט וסחיפת רגישות:** מד התאוצה ב MPU6050-עלול לסבול מהיסט וסחיפת רגישות עם הזמן ושינויי טמפרטורה. זה יכול לגרום למדידות לסטות מהערכים האמיתיים גם כשהמכשיר במנוחה. ניתן להפחית את השפעות הסחף באמצעות כיול מד התאוצה או טכניקות פיצוי מתאימות.
- (2) **רעש:** מד התאוצה רגיש לרעש, כולל הפרעות חשמליות ורעידות מכניות, שיכולות לגרום לתנודות אקראיות בקריאות ולפגוע בדיוק המדידות. טכניקות סינון או עיבוד אותות עשויות להידרש כדי להפחית את השפעת הרעש.
- (3) **טווח דינמי מוגבל:** טווח התאוצה הניתנת למדידה על ידי מד התאוצה מוגבל. חריגה מהטווח המרבי עלולה להוביל לרוויה בקריאות, ולאובדן מידע על תאוצות גבוהות יותר. ייתכן שיהיה צורך בבחירה קפדנית של טווח המדידה עבור יישומים מסוימים.
- (4) **תלות כיוון:** מד התאוצה מודד תאוצה ביחס לכוח הכבידה, ולכן מדידותיו מושפעות מכיוון המכשיר בחלל. כיוון לא נכון עלול לגרום לקריאות לא מדויקות. ייתכן שיהיה צורך באלגוריתמים לפיצוי ויישור נכון כדי לקחת בחשבון את השפעות הכיוון.

מה טוב לגבי מד התאוצה?

מד התאוצה ב MPU6050-אינו צובר הטיית מדידה לאורך זמן כמו הג'ירוסקופ. הוא מספק מדידות תאוצה מדויקות עם רעש גאומטרי בעל ממוצע אפס, מה שמבטיח שהשגיאה אינה מצטברת.



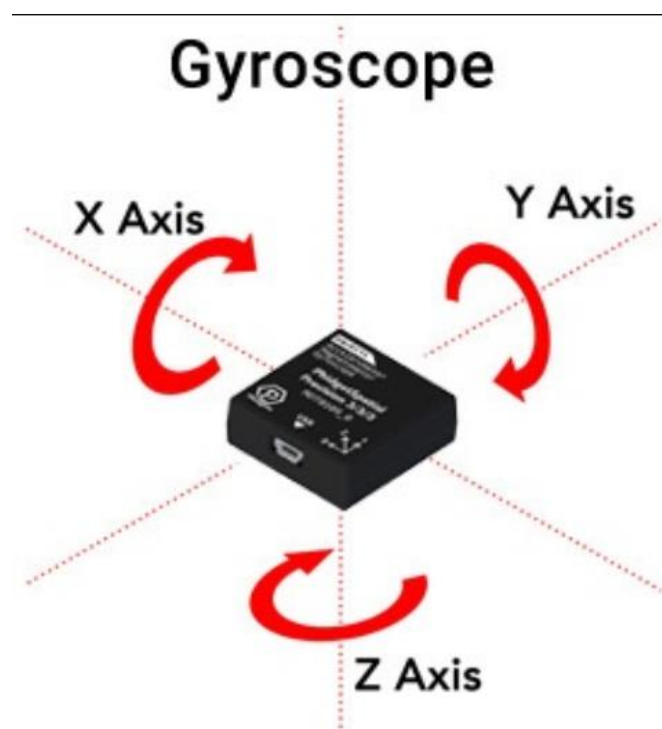
בעיות בג'ירוסקופ

לרכיב גירוסקופ MPU6050, כמו לכל רכיב אלקטרוני, יש מגבלות מסוימות ובעיות פוטנציאליות שהשתמשים צריכים להיות מודעים להן. להלן כמה אתגרים נפוצים הקשורים לרכיב הג'ירוסקופ של MPU6050:

1. **סחיפה:** ג'ירוסקופים רגישים לשינויים סביבתיים, שינויי טמפרטורה והזדקנות רכיבים, מה שעלול לגרום לסחיפה במדידות לאורך זמן, גם כשהמכשיר אינו מסתובב. סחיפה זו מצריכה קליברציה או טכניקות פיצוי כדי לשמור על דיוק המדידות.
2. **רעש:** קריאות הג'ירוסקופ עלולות להכיל רעש ותנודות אקראיות הנגרמות מהפרעות חשמליות, רעידות מכניות או פגמים בחיישן. רמות רעש גבוהות משפיעות על הדיוק וייתכן שיהיה צורך בטכניקות סינון או עיבוד אותות לשיפור איכות הנתונים.
3. **רגישות צולבת צירים:** הג'ירוסקופ מודד תנועה סיבובית לאורך שלושה צירים (X,Y,Z) אך יכולה להיות רגישות לציר צולב, שבה תנועה סיבובית בציר אחד עשויה להציג מדידות קלות בצירים אחרים. רגישות זו יכולה להשפיע על דיוק הקריאות, במיוחד כאשר נדרש מעקב מדויק אחר תנועה.
4. **קליברציה:** נדרשת קליברציה להפחתת הסחף ולשיפור הדיוק. תהליך הקליברציה כולל קביעת הטיות וקיצוזים במדידות והשימוש בהפניות חיצוניות לתיקון הנתונים.

יתרונות הג'ירוסקופ

הג'ירוסקופ ב-MPU6050 מתאפיין ברגישות גבוהה, המאפשרת לזהות שינויים קטנים במהירות זוויתית. רגישות זו חיונית ליישומים הדורשים מעקב תנועה מדויק והיענות מהירה.

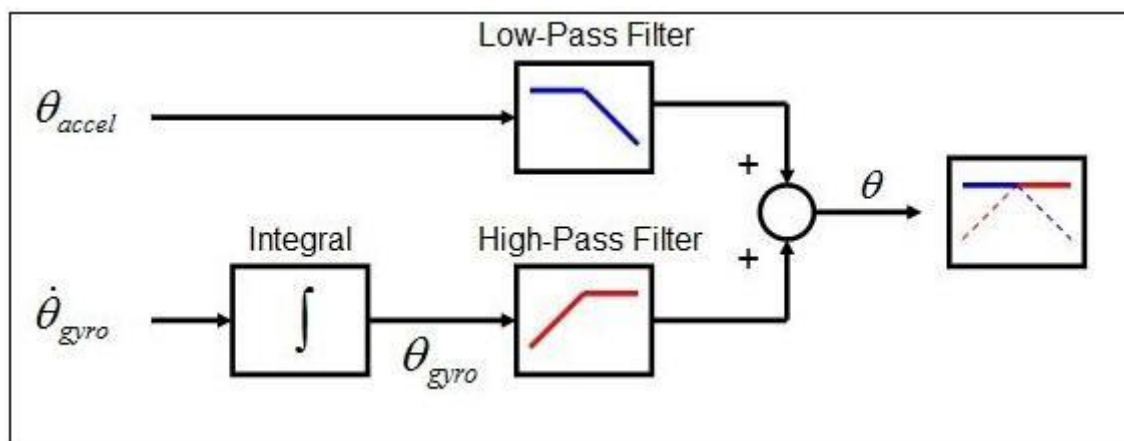


Complementary Filter

הפילטר המשלים הוא טכניקה לשילוב מידע ממספר חיישנים כדי לקבל אומדן מדויק של כמות פיזית, כמו זווית של אובייקט במרחב. בהקשר של ה-MPU6050-חיישן פופולרי המשלב מד תאוצה וג'ירוסקופ העובדים יחד בשישה צירים, ניתן להשתמש בפילטר זה להערכת הזווית של עצם באמצעות הנתונים משני החיישנים. הפילטר מבוסס על ממוצע משוקלל של הערכות הזווית שמתקבלות מכל חיישן. באופן ספציפי, הפילטר משתמש ב low-pass filter כדי לחלץ את רכיבי התדר הנמוך של נתוני מד התאוצה, ולשלב אותם עם רכיבי התדר הגבוה של נתוני הג'ירוסקופ כדי להגיע לאומדן זווית מדויק. ניתן להתאים את פרמטרי הפילטר כדי לשפר את ביצועיו ליישומים ספציפיים.

יתרונות הפילטר המשלים:

1. **דיוק משופר:** הפילטר מספק אומדן מדויק יותר של הזווית על ידי שילוב הנתונים משני החיישנים. מד התאוצה מדויק יותר בתדרים נמוכים, בעוד שהג'ירוסקופ מדויק יותר בתדרים גבוהים. הפילטר מנצל את העוצמות של שני החיישנים כדי לספק אומדן מדויק על פני טווח תדרים רחב יותר.
2. **הפחתת סחיפה:** ג'ירוסקופים נוטים לסבול מסחיפה לאורך זמן, מה שעלול לגרום לשגיאות בהערכת הזווית. הפילטר המשלים מפצה על הסחיפה באמצעות נתוני מד התאוצה, מה שמוביל לאומדן יציב יותר של הזווית לאורך זמן.
3. **דרישות חישוב נמוכות יותר:** הפילטר המשלים הוא אלגוריתם פשוט יחסית שדורש פחות כוח חישוב משיטות אחרות להערכת זווית, כמו ה-Kalman filter. זה הופך אותו למתאים במיוחד לשימוש במערכות משובצות או יישומים אחרים שבהם משאבי החישוב מוגבלים.
4. **יישום קל:** הפילטר המשלים הוא אלגוריתם פשוט ליישום בתוכנה או בחומרה, אפילו למי שיש לו ניסיון מוגבל בעיבוד אותות או בתיאוריית הבקרה.



חישוב זוויות עבור הפילטר המשלים ודוגמא

$$\theta_{acc} (\alpha - 1) + \alpha(\theta_{prev} + \Delta t \cdot \omega) = \theta$$

- θ היא הזווית המשוערת.
- Δt הוא הזמן בין הדגימות.
- ω היא המהירות הזוויתית שנמדדת על ידי הג'ירוסקופ.
- θ_{acc} היא הזווית המוערכת על ידי נתוני מד התאוצה.
- α הוא פרמטר כוונון בין 0 ל-1, הקובע את התרומה של נתוני הג'ירוסקופ ומד התאוצה לאומדן הכולל.

דוגמה לחישוב:

נניח שיש לנו את הנתונים הבאים:

- נתוני מד תאוצה: $\theta_{acc}=30^\circ$
- נתוני ג'ירוסקופ: $\omega=10^\circ/\text{sec}$
- זמן דגימה: $\Delta t=0.01 \text{ sec}$
- פרמטר כוונון: $\alpha=0.98$

חישוב המהירות הזוויתית מנתוני הג'ירוסקופ:

$$\omega_{est} = 10 \cdot 0.01 = 0.1^\circ$$

חישוב זווית הכיוון מנתוני מד התאוצה:

$$\theta_{acc} = \frac{180}{\pi} \cdot \text{atan2}(-a_y, a_z) = \frac{180}{\pi} \cdot \text{atan2}(-0.5, 0.866) = 30^\circ$$

שימוש במשוואת הפילטר המשלים לחישוב אומדן הכיוון הסופי:

$$\theta = \alpha(\theta + \Delta t \cdot \omega_{est}) + (1 - \alpha) \cdot \theta_{acc}$$

$$\theta = 0.98 \cdot (0 + 0.01 \cdot 0.1) + 0.02 \cdot 30$$

$$\theta = 0.001 + 0.6$$

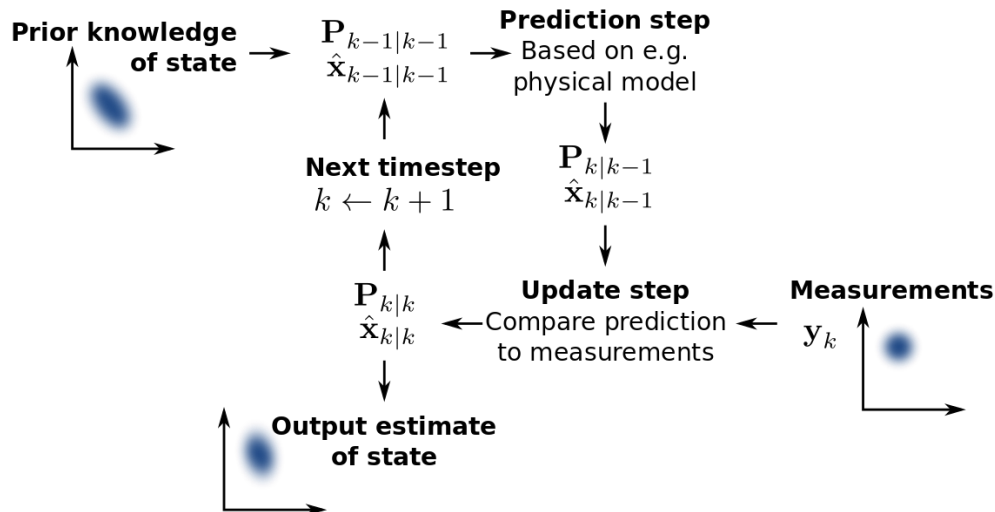
$$\theta = 0.601^\circ$$

זוהי ההערכה הסופית של זווית הכיוון. ניתן לחזור על תהליך זה עבור כל מדגם כדי לקבל אומדן רציף של הכיוון לאורך זמן. ניתן לכוון את פרמטר הכוונון α כדי לשלוט בתרומת נתוני הג'ירוסקופ ומד התאוצה לאומדן הסופי. ערך גבוה יותר של α ייתן משקל רב יותר לנתוני הג'ירוסקופ, בעוד שערך נמוך יותר ייתן משקל רב יותר לנתוני מד התאוצה.

שיטות חלופיות Kalman Filter

Kalman Filter הוא אלגוריתם רב עוצמה להערכת מצב מערכת על סמך מדידות חיישנים רועשים. הוא משתמש במודל הסתברותי של המערכת ומדידות החיישנים לאומדן מדויק יותר של זוויות. למרות שמסנן קלמן מורכב ודורש יותר משאבי חישוב, הוא יכול לספק הערכות מדויקות יותר בתנאים מסוימים.

המסנן שומר על אומדן של מצב המערכת ואומדן של אי הוודאות של אותו אומדן, המתעדכן על סמך מדידות חיישנים חדשות. הוא מתאים ליישומים שבהם נדרש דיוק גבוה ומשאבי חישוב זמינים.



השוואה בין הפילטר המשלים ו-Kalman Filter

- מורכבות:** מסנן קלמן מורכב יותר ודורש יותר משאבי חישוב ויותר קוד ליישום. הוא גם גמיש יותר ויכול להתמודד עם מערכות מורכבות יותר.
- דיוק:** מסנן קלמן מדויק יותר מהפילטר המשלים, במיוחד במצבים שבהם קריאות החיישנים רועשות מאוד. הוא לוקח בחשבון את קריאת החיישן הנוכחי, הדינמיקה של המערכת והערכות קודמות.
- עמידות:** הפילטר המשלים עמיד יותר ממסנן קלמן, במיוחד במצבים שבהם קריאות החיישנים יציבות יותר. הוא מסתמך על טכניקות סינון פשוטות שמושפעות פחות משינויים קטנים בקריאות החיישנים.
- קליברציה:** מסנן קלמן דורש ידע מדויק על מאפייני הרעש של החיישן והדינמיקה של המערכת, מה שמאתגר יותר לקליברציה. הפילטר המשלים אינו דורש קליברציה כה מדויקת וקל יותר להגדרה.
- שימוש במשאבים:** מסנן קלמן דורש יותר כוח עיבוד וזיכרון מאשר הפילטר המשלים, ולכן מתאים פחות ליישומים מוגבלים במשאבים.

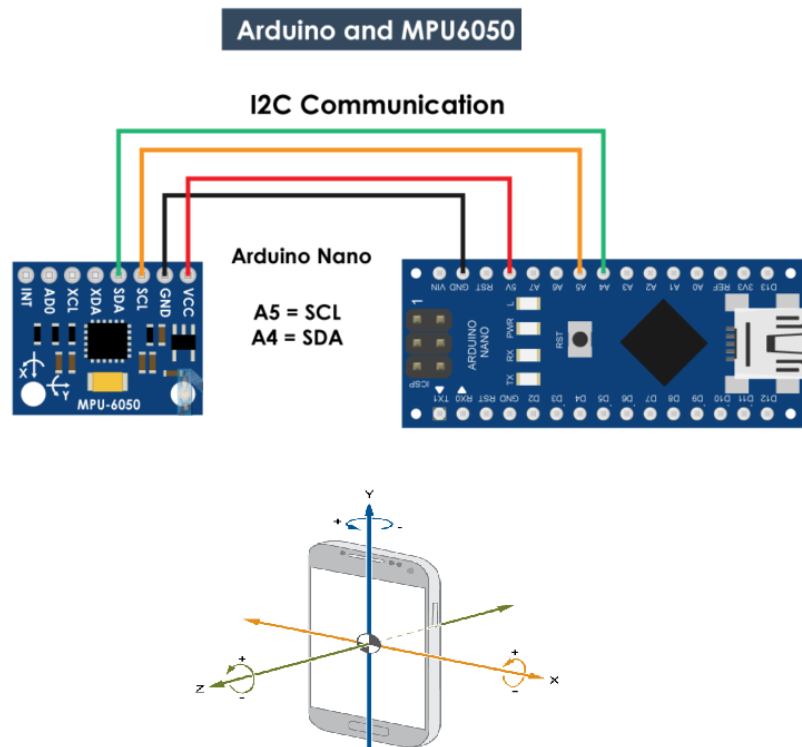
בסך הכל, הבחירה בין הפילטרים תלויה בדרישות היישום: אם נדרש דיוק גבוה ומשאבים זמינים, מסנן קלמן הוא הבחירה הטובה יותר. אם החוסן והפשטות חשובים יותר, הפילטר המשלים הוא הבחירה המתאימה.

צורת התקשרות ב-MPU6050

הרכיב MPU6050 מתקשר עם ה-Arduino Nano באמצעות פרוטוקול I2C.

I2C – פרוטוקול (**I2C (Inter-Integrated Circuit)** הוא פרוטוקול תקשורת טורי דו-חוטי המאפשר למספר מכשירים לתקשר זה עם זה. פרוטוקול זה פותח בשנות ה-80 על ידי חברת פיליפס (כיום-NXP Semiconductors) כפתרון לחיבור רכיבים שונים בטלויזיה.

פרוטוקול I2C מבוסס על שני קווים **SDA**: (קווי נתונים) ו **SCL**-(קווי שעון). קו ה **SDA**-אחראי להעברת הנתונים בין המכשירים, בעוד שקו ה **SCL**-משמש לסנכרון העברת הנתונים ביניהם.

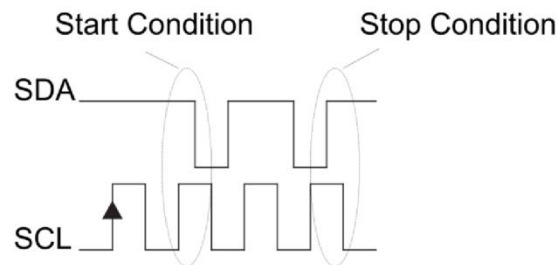


במערכת I2C, אחד המכשירים מוגדר כמאסטר והאחרים כעבדים. המאסטר יוזם את כל התקשורת ומייצר את אות השעון להעברת הנתונים. העבדים מגיבים לפקודות ולנתונים שנשלחים על ידי המאסטר.

כדי להתחיל תקשורת, המאסטר שולח אות התחלה (מעבר מרמה נמוכה לגבוהה ב SDA-כאשר SCL גבוה) ולאחר מכן שולח את כתובת העבד בת 7 הסיביות שאליו הוא רוצה לפנות, יחד עם סיבית קריאה/כתיבה לציון כיוון העברת הנתונים. העבד עם הכתובת המתאימה מגיב באות אישור (פולס נמוך ב SDA-במהלך פעימת השעון התשיעי), (וכך מתחילה התקשורת).

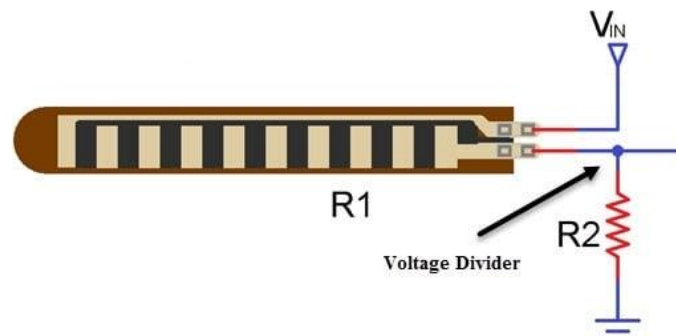
לאחר יצירת התקשורת, המאסטר שולח פקודות או נתונים לעבד על ידי שליחת בייתי נתונים ואות אישור מהעבד. העבד יכול גם לשלוח נתונים למאסטר באמצעות הנעת קו SDA כאשר המאסטר מספק את אות השעון. כאשר התקשורת מסתיימת, המאסטר שולח אות עצירה) מעבר מרמה גבוהה לנמוכה ב SDA-כאשר SCL גבוה) (כדי לסמן את סיום התקשורת.

I2C הוא פרוטוקול פופולרי לחיבור חיישנים, מפעילים והתקנים אחרים במערכות משובצות, כיוון שהוא מאפשר לחבר מספר התקנים למיקרו-בקר באמצעות שני חוטים בלבד. הפרוטוקול איטי יחסית, עם קצבי העברת נתונים שנעים בין 100 קילו-ביט לשנייה ועד 5 מגה-ביט לשנייה, בהתאם לגרסת הפרוטוקול ולמכשירים בשימוש. למרות זאת, הוא פשוט וקל ליישום, מה שהופך אותו לבחירה פופולרית עבור יישומים רבים.



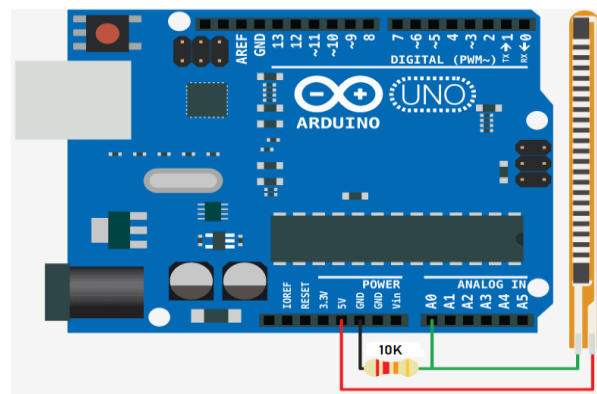
הזזת המנועים על ידי Flex Sensor

חיישן גמיש או עיקול הוא חיישן שמודד את מידת הסטייה או הכיפוף. בדרך כלל, החיישן דבוק למשטח, וההתנגדות של אלמנט החיישן משתנה על ידי כיפוף המשטח. מכיוון שההתנגדות עומדת ביחס ישר לכמות העיקול, היא משמשת כגוניומטר, ולעתים קרובות נקרא פוטנציומטר גמיש.



ככל שנכוף את האצבע שהחיישן נמצא עליה הערך האנלוגי שלה ישתנה ונוכל להציב סוג של threshold מסוים כשהיא תעבור אותו או תהיה בתווך מסוים שנציב לו היא תשלח לזרוע פקודה שתזיז את המונע למקום הרצוי.

בדרך כלל נחבר לחיישן נגד בגודל 10 קילו אוהם ושני רגלים מחוברים אחת ל-VCC ואחת ל-GND שגם מחובר ל-pin אנלוגי בצורה הבאה:



חישוב מיקום ה-flex sensor והמרה קריאה אנלוגית לדיגיטלית

בגלל שה-flex sensor הוא סוג של נגד משתנה, בגלל החיבור שלו נוכל להראות את חישובים של מחלק מתח ולגלות את ההתנגדות שלו בזווית כיפול מסוימת.

תחילה נראה איך להפוך קריאה דיגיטלית למתח

$$\frac{v_i}{v_{ref}} = \frac{D_i}{2^N} \Rightarrow v_i = \frac{D_i \cdot v_{ref}}{2^N}$$

$$\left[\begin{array}{l} v_{ref} = 5v \\ N = 10 \end{array} \right]$$

$$\textcircled{1} \quad v_i = \frac{5 \cdot D_i}{1024}$$

בארדואינו במקום 1024 יהיה 1023 בגלל שהיא אינטואיטיבית וההבדל בסוף זניה

1

נשתמש בנוסחה של מחלק המתח

$$v_{in} = I_T \cdot (R + R_{Flex})$$

$$I_T = \frac{v_{in}}{R + R_{Flex}}$$

מכאן שהמתח הנופל על הנגד הוא

$$V_o = V_R = I_T \cdot R$$

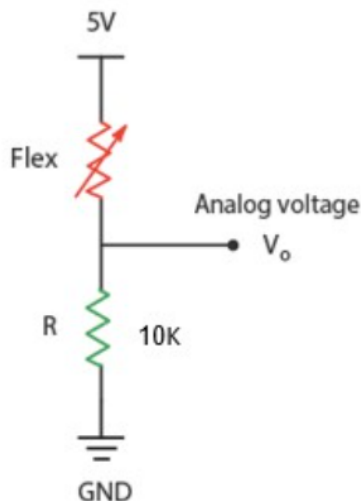
$$V_o = \frac{v_{in}}{R + R_{Flex}} \cdot R$$

$$V_o = \frac{R}{R + R_{Flex}} \cdot v_{in}$$

$$\left[\begin{array}{l} v_{in} = 5v \\ R_1 = 10K\Omega \end{array} \right]$$

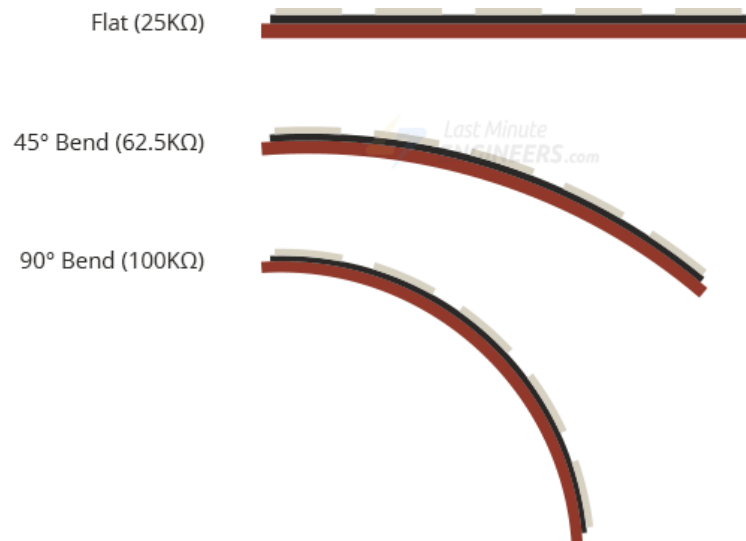
$$\textcircled{2} \quad V_o = \frac{5 \cdot 10000}{10000 + R_{Flex}}$$

2



ספר פרויקט במערכות אוטונומיות – דרוע רובוטית שנשלטת באמצעות כפפה – גלב שורין

אחרי שנקבל את הערכים נקבע לפי התמונה הבאה את זווית הנגד לפי ההתנגדות בהתאמה ונעשה את זה לכל אצבע בנפרד בהתאמה לפי הזווית המתאימה.



קוד דוגמה שממיר את הקריאה האנלוגי של הנגד לאות דיגיטלי ומקבל את המתח עליו ומחשב את הזווית של הנגד הגמיש בהתאמה:

```
1 const int flexPin = A0;           // Pin connected to voltage divider output
2
3 // Change these constants according to your project's design
4 const float VCC = 5;              // voltage at Arduino 5V line
5 const float R_DIV = 47000.0;      // resistor used to create a voltage divider
6 const float flatResistance = 25000.0; // resistance when flat
7 const float bendResistance = 100000.0; // resistance at 90 deg
8
9 void setup() {
10     Serial.begin(9600);
11     pinMode(flexPin, INPUT);
12 }
13
14 void loop() {
15     // Read the ADC, and calculate voltage and resistance from it
16     int ADCflex = analogRead(flexPin);
17     float Vflex = ADCflex * VCC / 1023.0;
18     float Rflex = R_DIV * (VCC / Vflex - 1.0);
19     Serial.println("Resistance: " + Rflex + " ohms");
20
21     // Use the calculated resistance to estimate the sensor's bend angle:
22     float angle = map(Rflex, flatResistance, bendResistance, 0, 90.0);
23     Serial.println("Bend: " + angle + " degrees");
24     Serial.println();
25
26     delay(500);
27 }
```

שליחת הנתונים לזרוע מהכפפה

ה-HC-05 הוא מודול Bluetooth המיועד לתקשורת אלחוטית. ניתן להשתמש במודול זה בתצורת מאסטר או עבד.

הוא משמש ליישומים רבים כמו אוזניות אלחוטיות, בקרי משחק, עכבר אלחוטי, מקלדת אלחוטית ועוד יישומים צרכניים רבים.

יש לו טווח פעולה של עד פחות מ-100 מטר, אשר תלוי במשדר ובמקלט, באטמוספירה, בתנאים גיאוגרפיים ובעירוניים.

זהו פרוטוקול סטנדרטי של IEEE 802.15.1 באמצעותו ניתן לבנות רשת אלחוטית אישית (PAN). הוא משתמש בטכנולוגיית רדיו של תדר-קפיצה (FHSS) כדי לשלוח נתונים באוויר.

הוא משתמש בתקשורת טורית כדי לתקשר עם מכשירים. הוא מתקשר עם המיקרו בקר באמצעות פורט טורי (UART).

מודולי Bluetooth טוריים מאפשרים לכל המכשירים התומכים בטורי לתקשר זה עם זה באמצעות Bluetooth.

יש לו 6 פינים,

1. **EN/Key**: הוא משמש להכנסת מודול בלוטות' למצב פקודות AT. אם ה-Pin Key/EN מוגדר לגבוהה, מודול זה יעבוד במצב פקודה. אחרת כברירת מחדל היא במצב נתונים. קצב ההחזרה המוגדר כברירת מחדל של HC-05 במצב פקודה הוא 38400bps ו-9600 במצב נתונים.
למודול HC-05 יש שני מצבים:
 1. מצב נתונים: חילופי נתונים בין מכשירים.
 2. מצב פקודה: הוא משתמש בפקודות AT המשמשות לשינוי הגדרות של HC-05. כדי לשלוח פקודות אלה ליציאת מודול טורית (UART).
2. **VCC**: לחבר 5 V או 3.3 V לפי זה.
3. **GND**: פין האדמה של המודול.
4. **TXD**: העברת נתונים סדרתיים (נתונים שהתקבלו באופן אלחוטי על ידי מודול Bluetooth משודרים באופן סדרתי על פין TXD)
5. **RXD**: קבלת נתונים באופן סדרתי (הנתונים שהתקבלו ישודרו באופן אלחוטי באמצעות מודול Bluetooth).
6. **State**: זה אומר אם המודול מחובר או לא.



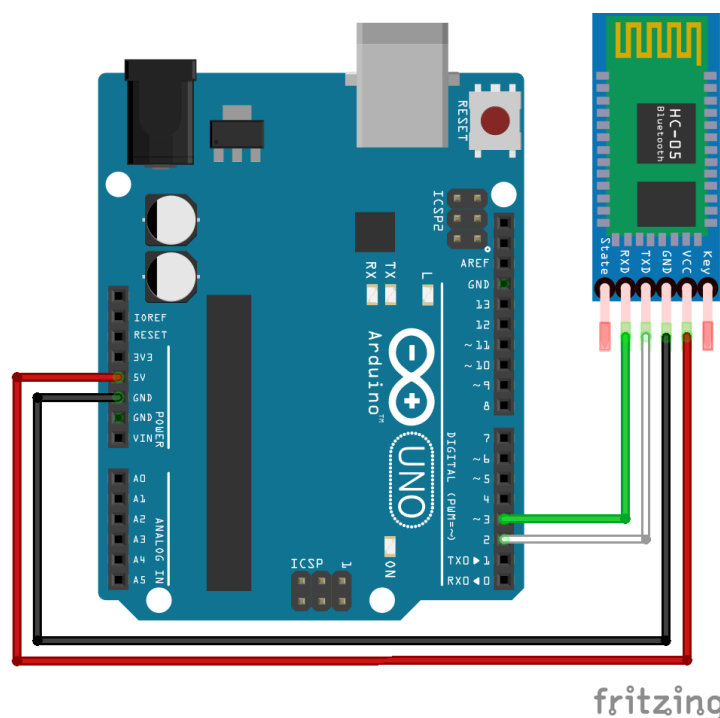
ספר פרויקט במערכות אוטונומיות – זרוע רובוטית שנשלטת באמצעות כפפה – גלב שורין

אני השתמשתי במודול זה בשביל להעביר את הנתונים שהכפפה שלי קולטת לזרוע הרובוטית שלי.

אנחנו לוקחים מודול אחד ובעזרת הפקודות של ה-AT מעבירים אותו למצב Master ומחברים אותו לכתובת של המודול השני של הזרוע שנקרא Slave, צורפה דוגמה לטבלה של מספר פקודות AT להדגמה:

Command	Description	Response
AT	Checking communication	OK
AT+PSWD=XXXX e.g. AT+PSWD=4567	Set Password	OK
AT+NAME=XXXX e.g. AT+NAME=MyHC-05	Set Bluetooth Device Name	OK
AT+UART=Baud rate, stop bit, parity bit e.g. AT+UART=9600,1,0	Change Baud rate	OK
AT+VERSION?	Respond version no. of Bluetooth module	+Version: XX OK e.g. +Version: 2.0 20130107 OK
AT+ORGL	Send detail of setting done by manufacturer	Parameters: device type, module mode, serial parameter, passkey, etc.

נוכל לחבר את המודול לבקרים שלנו שהם ה-Arduino Uno וה-Arduino Nano בצורה הבאה:



fritzing

ספר פרויקט במערכות אוטונומיות – זרוע רובוטית שנשלטת באמצעות כפפה – גלב שורין

חשוב לציין שבגלל הבעיה שיש בבקרים של Arduino Uno כאשר ה-TX וה-RX תפוסים לא ניתן להעלות קוד לבקר אז השתמשנו בספריה של Arduino IDE שנקרא Software Serial שמאפשר לנו לשנות להשתמש בפינים אחרים של הבקר בתור ה-RX וה-TX שלנו.

ולפי האתר הרשמי של Arduino הספרייה תוארה כך:

ספריית SoftwareSerial מאפשרת תקשורת טורית על פינים דיגיטליים אחרים של לוח Arduino, תוך שימוש בתוכנה לשכפול הפונקציונליות (ומכאן השם "SoftwareSerial"). אפשר לקבל מספר יציאות טוריות של תוכנה עם מהירויות של עד 115200 bps. פרמטר מאפשר איתות הפוך עבור התקנים הדורשים פרוטוקול זה.

דוגמה לקוד שמשמש בספרייה:

```
1  #include <SoftwareSerial.h>
2
3  #define rxPin 10
4  #define txPin 11
5
6  // Set up a new SoftwareSerial object
7  SoftwareSerial mySerial = SoftwareSerial(rxPin, txPin);
8
9  void setup() {
10     // Define pin modes for TX and RX
11     pinMode(rxPin, INPUT);
12     pinMode(txPin, OUTPUT);
13
14     // Set the baud rate for the SoftwareSerial object
15     mySerial.begin(9600);
16 }
17
18 void loop() {
19     if (mySerial.available() > 0) {
20         mySerial.read();
21     }
22 }
```

*השתמשי בספרייה רק בחלק של הזרוע, הכפפה עדיין מחוברת כרגיל.

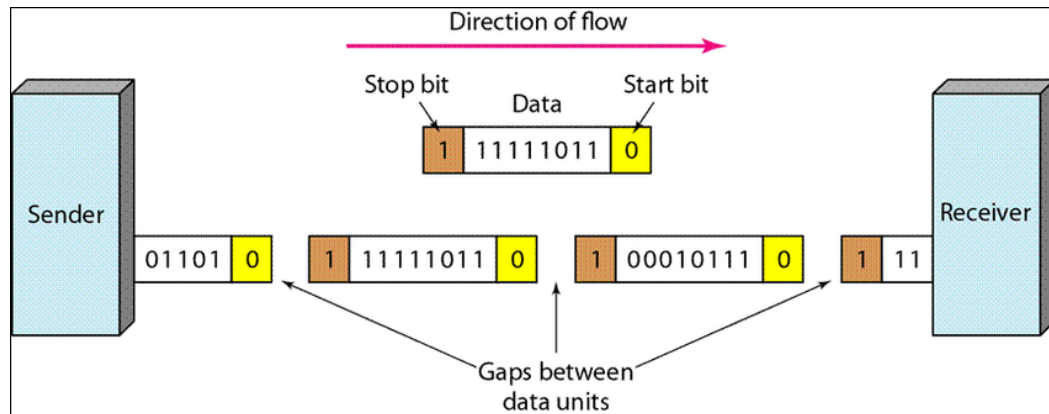
המודול משתמש בתקשורת טורי (Serial communication), תקשורת טורית היא פרוטוקול המשמש מיקרו-בקרים להעברת וקבלת נתונים בין שני מכשירים. מדובר בשיטה פשוטה ואמינה להחלפת נתונים, פקודות ומידע אחר בין מערכות דיגיטליות.

בתקשורת טורית, הנתונים נשלחים ומתקבלים ביט אחר ביט בזרם רציף, בניגוד לתקשורת מקבילה שבה מספר ביטים נשלחים בו-זמנית על פני קווים נפרדים.

במיקרו-בקרים, תקשורת טורית ניתנת ליישום באמצעות פרוטוקולים מבוססי חומרה או תוכנה. הפרוטוקולים הטוריים הנפוצים ביותר כללים UART, SPI ו-I2C.

ספר פרויקט במערכות אוטונומיות – זרוע רובוטית שנשלטת באמצעות כפפה – גלב שורין

UART הוא הפרוטוקול הטורי הפשוט והנפוץ ביותר במיקרו-בקרים. הוא משתמש בשני חוטים לתקשורת: אחד להעברת נתונים (TX) ואחד לקבלת נתונים (RX). הנתונים מועברים עם סיביות התחלה ועצירה, ובמקרים מסוימים גם עם סיביות זוגיות, כדי להבטיח את שלמות הנתונים.



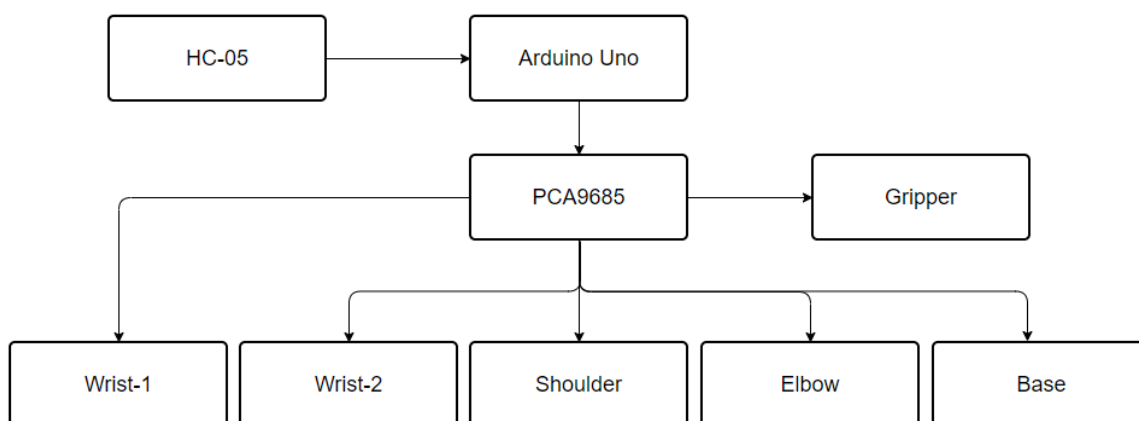
לסיכום, תקשורת טורית היא מרכיב קריטי במערכות מיקרו-בקר, המספקת דרך פשוטה ויעילה להעברת נתונים ופקודות בין מכשירים. באמצעות פרוטוקולים טוריים שונים, מיקרו-בקרים יכולים לתקשר עם מגוון רחב של התקנים היקפיים, מה שמאפשר להם לבצע מגוון רחב של פונקציות ויישומים.

הזרוע הרובוטית

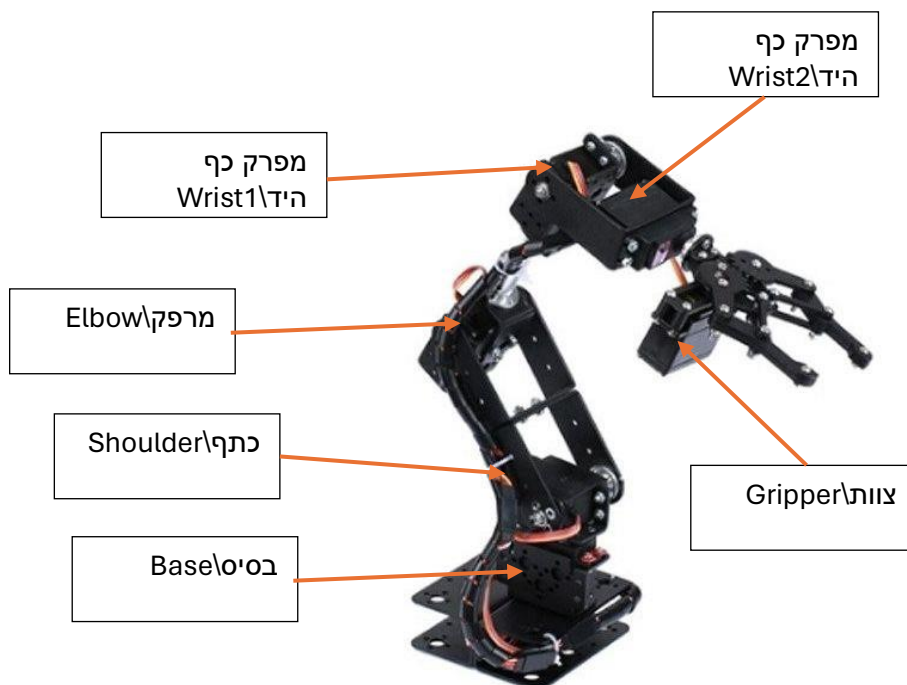
הזרוע הרובוטית הרבה יותר קלה למימוש, הבנייה שלה יותר קשה מהכפפה אבל הארכיטקטורה הרבה יותר קלה להבנה.

חלקי הזרוע ופעולותיה

אלו החלקים הנדרשים בשביל להרכיב את כל הזרוע:



כל מנוע בזרוע אחראי למשהו וקוראים להם בשמות הנה תמונה המתארת את כולם:



מנועי היד ושליטתם

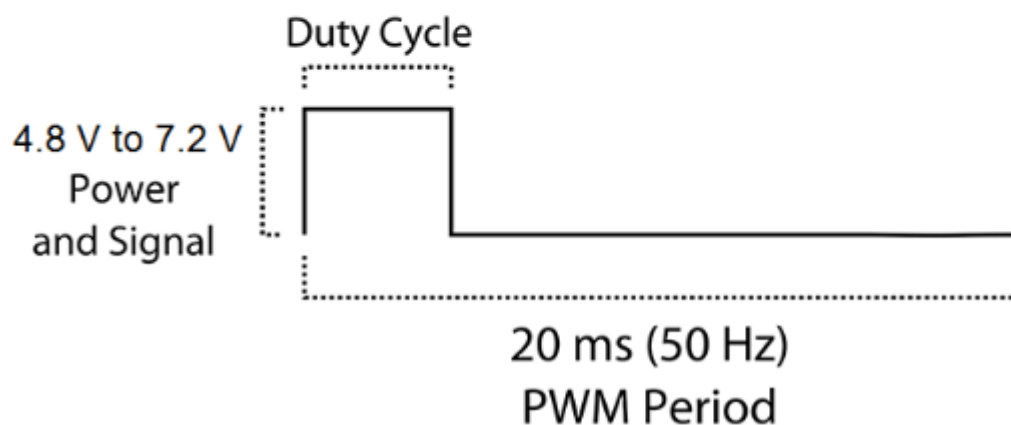
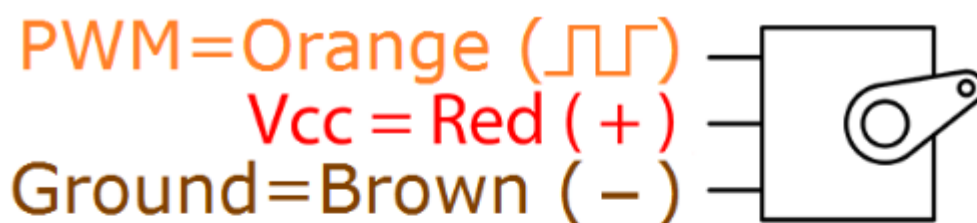
בזרוע יש 6 מנועים מסוג Servo גרסה MG996R שאלו מנועים שהם גרסה משודרגת של המנוע MG995.

למנוע יש PCB חדש והמערכת שליטה IC חדשה שהופכת אותו ליותר מדויק.

מפרט המנוע

- Weight: 55 g
- Dimension: 40.7 x 19.7 x 42.9 mm approx.
- Stall torque: 9.4 kgf·cm (4.8 V), 11 kgf·cm (6 V)
- Operating speed: 0.17 s/60° (4.8 V), 0.14 s/60° (6 V)
- Operating voltage: 4.8 V a 7.2 V
- Running Current 500 mA – 900 mA (6V)
- Stall Current 2.5 A (6V)
- Dead band width: 5 μ s
- Stable and shock proof double ball bearing design
- Temperature range: 0 °C – 55 °C

כפי שניתן לראות למטה ה-PWM Period הוא 20ms מה שאומר שאפשר לשלוח pulse כל 20 מילי שניות.

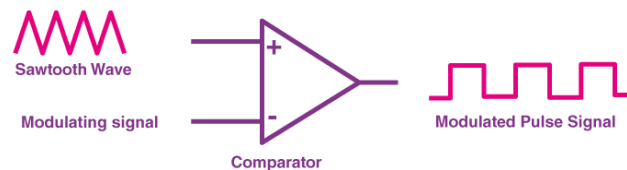


הסבר על האות PWM במנועים

המנוע עובד עם אות מסוג PWM.

PWM או Pulse Width Modulation מפחית את ההספק הממוצע הנמסר על ידי המרת האות לחלקים נפרדים. בטכניקת PWM, אנרגיית האות מופצת באמצעות סדרה של pulses במקום אות משתנה ברציפות (אנלוגי).

ה-PWM נוצר באמצעות משווה (comparator). ה-modulation signal מהווה חלק אחד מהקלט למשוואה, בעוד שה-non-sinusoidal wave או ה-sawtooth wave יוצר את החלק השני של הקלט. המשוואה משווה שני אותות ומייצר את PWM כצורת גל הפלט שלו.



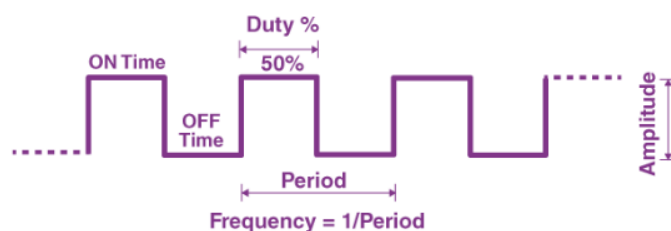
אם ה-sawtooth signal הוא יותר מ-modulation signal, אזי האות הפלט נמצא במצב "גבוה". ערך הגודל קובע את פלט ההשוואה המגדיר את רוחב הפולס שנוצר במוצא.

ה-Duty Cycle של ה-PWM עובד כך, אנחנו יודעים כי ה-PWM signal נשאר "ON" למשך זמן נתון ונשאר "OFF" למשך זמן מסוים. אחוז הזמן שבו האות נשאר "ON" ידוע בתור duty cycle. אם האות תמיד "ON", אזי האות חייב להיות בעל duty cycle של 100%. הנוסחה לחשוב מחזור העבודה היא:

$$Duty Cycle = \frac{Turn On Time}{Turn On Time + Turn Off Time}$$

הערך הממוצע של המתח תלוי במחזור העבודה. כתוצאה מכך, ניתן לשנות את הערך הממוצע על ידי שליטה על רוחב ה-"ON" של ה-pulse.

ה-frequency של ה-PWM:



התדירות של ה-PWM קובעת כמה מהר ה-PWM משלים period. תדירות ה-pulse מוצגת באיור שלמעלה.

ניתן לחשב את התדירות של ה-PWM באופן הבא:

תדירות = $1/Time Period$.

ה- $On Time + Off Time = Time Period$.

ספר פרויקט במערכות אוטונומיות – זרוע רובוטית שנשלטת באמצעות כפפה – גלב שורין

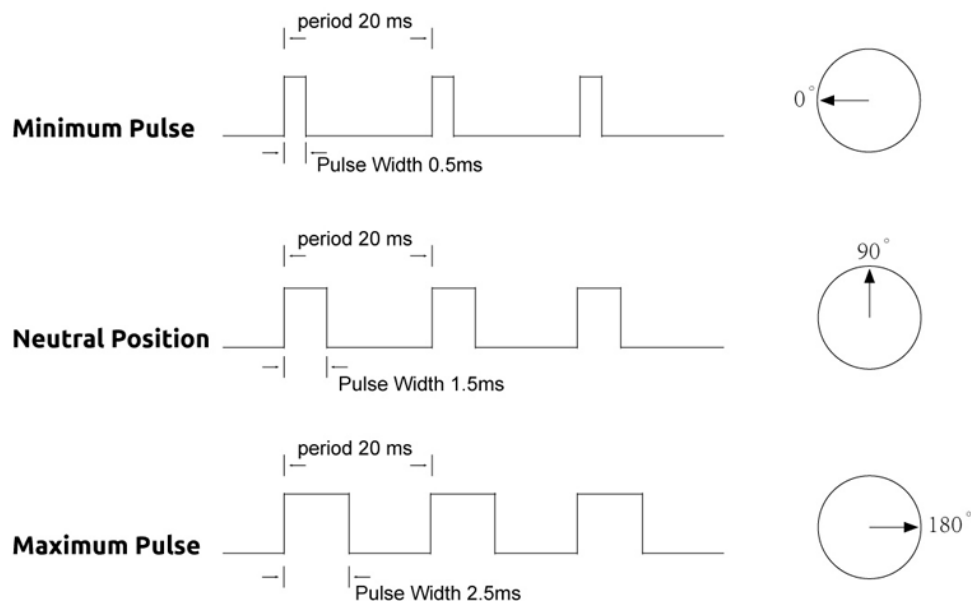
מתח המוצא של אות ה-PWM יהיה האחוז של מחזור העבודה. לדוגמה, עבור מחזור עבודה של 100%, אם מתח הפעולה הוא 5 וולט אז מתח המוצא יהיה 5 וולט. אם מחזור העבודה הוא 50%, אז מתח המוצא יהיה 2.5 וולט.

ה-Servo נשלטים על ידי שליחת ה-PWM שהוסבר קודם לכן. יש pulse מינימלי ויש pulse מקסימלי, ו-repetition rate. מנוע Servo יכול בדרך כלל להסתובב רק 90 מעלות לכל כיוון עבור תנועה כוללת של 180 מעלות.

המיקום הניטרלי של המנוע מוגדר כמצב שבו ל-servo יש אותה כמות של סיבוב פוטנציאלי גם בכיוון השעון וגם נגד כיוון השעון.

כדי להבין טוב יותר מהו מנוע Servo, ה-PWM הנשלח למנוע קובע את מיקום הציר, ועל סמך משך הפולס הנשלח דרך חוט הבקרה, הרוטור (rotor) יפנה למצב הרצוי.

מנוע ה-Servo מצפה לראות pulse כל 20 מילישניות (ms), ואורך ה-pulse קובע עד כמה המנוע מסתובב. הנה דוגמה – pulse של 1.5ms יגרום למנוע להסתובב למצב של 90 מעלות. קצר מ-1.5ms מזיז אותו נגד כיוון השעון לעבר מיקום ה-0 מעלות. ויותר מ-1.5ms יפנה את ה-Servo בכיוון השעון לעבר מיקום ה-180 מעלות.



כאשר ה-Servo האלו יקבלו פקודה לנוע, הם יעברו לעמדה ויחזיקו בעמדה זו, אם כוח חיצוני דוחף אל ה-Servo בזמן שה-Servo יתנגד מלצאת מעמדה זו. כמות הכוח המקסימלי שה-Servo יכול להפעיל נקראת דירוג המומנט של ה-Servo. עם זאת, Servo לא יחזיקו בעמדם לנצח: יש לחזור על pulse המיקום כדי להורות ל-Servo להישאר במצב.

הסיפרה שאני משתמש בה Adafruit_PWMServoDriver אומרת שאורך ה-pulse המקסימלי והמינימלי (מתוך 4096 מילישניות) הם 600 ו-100 בהתאמה.

נראה במהלך הפרויקט שאני במקום pulse השתמשתי בזווית בשביל להזיז את המנוע, זאת אומרת בניתי סוג של קו ישר שממיר מזווית ל-pulse וזה נשלח ל-מנוע.

הסוללה לששת המנועים

אפשר לראות שאם כל מנוע צורך מקסימום 7.2v והזרם הוא 900 מילי אמפר נראה כי סוללה של 7.2v ב-4200mah תהיה מושלם ואידיאלית ביותר לשעה של עבודה עם הזרוע.

עקב אילוצים של הבנייה קיבלתי סוללה יותר חזקה ממה שצריך ובשביל לאזן את המתח השתמשתי במייצב מתח מסוג LM317 וייצבתי את המתח ביציאה להיות קבוע בין 6 וולט ו-7.2 וולט, כמובן כל עוד אנחנו משתמשים בסוללה שיש לי ספציפית שהיא 8 וולט ו-4200mah.

כמובן הכי חשוב מכל הוא כמה כל מנוע יכול לזוז, המנועים שאני קניתי מוגבלים לתזוזה של עד 180 מעלות כל אחד כך שיהיה תנועה מספיקה אבל לא יתר על המידה כמו שהיה קורה אם הייתי בוחר במנוע בעל ציר סיבובי של 360 מעלות.



בקר השליטה במנועים

מודול PCA9685 הוא יציאת PWM (Pulse Width Modulation) מרובת ערוצים, המופעל דרך פרוטוקול ה-I2C. הוא מסוגל לקבוע את רוחב הפולס של עד 16 ערוצים באופן עצמאי, ולכן הוא נפוץ בשליטה במנועי סרוו, מנועים חשמליים, ומקורות אור LED.

אני השתמשתי בו כי אני צריך לשלוט ב-6 מנועים חלקם במקביל ואין לי מספיק פורטים בבקר ה-Arduino Uno לשלוט בזה.

המודול מושלם לעבודה ופשוט להבנה, ישנה ספרייה שלו באינטרנט ואף ב-Arduino IDE בשם Adafruit_PWMServoDriver הוא נותן לי דרך קלה לשלוט ב-pulse שכל מנוע מקבל.



תקשורת וקבלת מידע מהכפפה

אני משתמש כאן במודול אלחוטי HC05 לקבל המידע מהכפפה, התקשורת והרכיב הוסברו כבר למעלה בפירוט, אכן אציין שבאן היה שימוש ב-software serial שדיברתי עליו לפני.

פונקציות עיקריות בפרויקט

פונקציות חשובות בכפפה

```

1 void getmpu1filter(){
2     // Read accelerometer and gyroscope data
3     sensors_event_t accel, gyro, temp;
4     mpu.getEvent(&accel, &gyro, &temp);
5
6     // Apply gyro calibration offsets
7     gyro.gyro.x -= PGYROOFFSETX;
8     gyro.gyro.y -= PGYROOFFSETY;
9     gyro.gyro.z -= PGYROOFFSETZ;
10
11    // Calculate angle from accelerometer data
12    pitch1 = atan2(-accel.acceleration.x, sqrt(pow(accel.acceleration.y, 2) + pow(accel.acceleration.z, 2)));
13    roll1 = atan2(accel.acceleration.y, accel.acceleration.z);
14
15    // Calculate change in time
16    unsigned long currentTime = millis();
17    float deltaTime = (currentTime - lastTime1) / 1000.0; // Convert to seconds
18    lastTime1 = currentTime;
19
20    // Calculate angle change from gyroscope data
21    float gyro_pitch = gyro.gyro.x * (deltaTime / MPU6050_GYRO_FS_SEL);
22    float gyro_roll = gyro.gyro.y * (deltaTime / MPU6050_GYRO_FS_SEL);
23    float gyro_yaw = gyro.gyro.z * (deltaTime / MPU6050_GYRO_FS_SEL);
24
25    // Combine angles using complementary filter
26    pitch1 = alpha * (pitch1 + gyro_pitch) + (1 - alpha) * pitch1;
27    roll1 = alpha * (roll1 + gyro_roll) + (1 - alpha) * roll1;
28    yaw1 += gyro_yaw;
29
30    // Add a small delay
31    delay(dt * 1000); // Convert to milliseconds
32 }

```

הפונקציה קוראת את המידע מחיישן ה-MPU6050, משיים את הפילטר המשלים בשביל לחשב את אוריינטציית הזוויות ה-pitch, roll וה-yaw, ואז מעדכן אותם עם הזמן.

- שורה 4-3: הפונקציה מגדירה את המשתנים (accel, gyro, temp) ושומר את המידע לגביי התאוצה, הג'ירו והטמפרטורה.
- שורה 7-9: השורות מחסירות את ה-calibration offsets שחישבנו לפני בשביל לתקן את ה-bias של ה-gyro בקריאות שלו.
- שורה 12-13: ה-pitch מחושב על ידי הפונקציה atan2 בשביל למצוא את הזווית בעזרת המידע על ציר ה-x של מד התאוצה וציר ה-y וציר ה-z. ה-roll מחושב בצורה דומה.
- שורה 16-18: מחשבים את ה-deltaTime וממירים לשניות.
- שורה 21-23: השורות מחשבות את השינוי בזוויות ה-pitch, roll וה-yaw בהתבסס על מידע ה-gyro והזמן שעבר (Delta Time), המשתנה הנוסף רק מדייק את הקריאה.
- שורה 26-28: הפילטר המשלים מחבר את המידע ממד התאוצה וה-gyro ונותן קריאה יציבה.
- שורה 31: השורה סה"כ נותן delay קטן.

```

1 void send_mpu_data(){
2   //Move Left
3   if ( (roll1 * 180 / M_PI) >= 40 && ((pitch1 * 180 / M_PI) >= -25 && (pitch1 * 180 / M_PI) <= 25)) {
4     Serial.print("L");
5     delay(response_time);
6   }
7
8   //Move Right
9   if ( (roll1 * 180 / M_PI) <= -80 && ((pitch1 * 180 / M_PI) >= -25 && (pitch1 * 180 / M_PI) <= 25)) {
10    Serial.print("R");
11    delay(response_time);
12  }
13
14  //Claw Up
15  if ( (pitch1 * 180 / M_PI) >= 45 ) {
16    Serial.print("U"); // G
17    delay(response_time);
18  }
19
20  //Claw Down
21  if ( (pitch1 * 180 / M_PI) <= -45 ) {
22    Serial.print("D"); // U
23    delay(response_time);
24  }
25
26  //Move to me
27  if ( pitch2 * 180 / M_PI >= 45) { //|| yaw1 * 180 / M_PI >= -0.17) {
28    Serial.print("C");
29    delay(response_time);
30  }
31
32  //Move from me
33  if ( pitch2 * 180 / M_PI <= -45) { // || yaw1 * 180 / M_PI <= -0.55
34    Serial.print("c");
35    delay(response_time);
36  }
37 }
38 }

```

הפונקציה שולחת את המידע לגבי מיקום הכפפה לזרוע והזרוע מזיזה את המנוע הרצוי.

- האות L תשלח לזרוע ומנוע Wrist2 יזוז סיבוב עם כיוון השעון.
- האות R תשלח לזרוע ומנוע Wrist2 יזוז נגד כיוון השעון.
- האות U תשלח לזרוע ומנוע Wrist1 יזוז למעלה (יזוז אחורה).
- האות D תשלח לזרוע ומנוע Wrist1 יזוז למטה (יזוז קדימה).
- האות C תשלח לזרוע ומנוע Shoulder יזוז לכיוון שלנו (יזוז אחורה).
- האות c תשלח לזרוע ומנוע Shoulder יזוז לכיוון ההפוך מאיתנו (יזוז קדימה).

```
1 void send_flex_data(){
2     // Pinkie - (Elbow)
3     if (pinkie <= pinkie_high && pinkie >= 250) {
4         Serial.print("P");
5         delay(response_time);
6     }
7
8     if (pinkie <= pinkie_low ) {
9         Serial.print("p");
10        delay(response_time);
11    }
12
13
14    // thumb 1 - thumb (Base Rotation)
15    if (thumb <= thumb_high) {
16        Serial.print("T");
17        delay(response_time);
18    }
19
20    if (thumb <= thumb_low && thumb >= 100) {
21        Serial.print("t");
22        delay(response_time);
23    }
24
25    // finger 1 - Claw Bend/Open
26    if (finger >= finger_high) {
27        Serial.print("F");
28        delay(response_time);
29    }
30
31    if (finger <= finger_low) {
32        Serial.print("f");
33        delay(response_time);
34    }
35    else {
36        delay(5);
37    }
38 }
```

הפונקציה שולחת את המידע לגבי כיפוף האצבע מהכפפה לדרוע ומזיזה את המנוע הרצוי.

- האות F תשלח בשביל לפתוח את הצוות או לפתוח את מנוע ה-Gripper.
- האות f תשלח בשביל לסגור את הצוות או לסגור את מנוע ה-Gripper.
- האות T תשלח ותזיז את מנוע ה-Base ימינה.
- האות t תשלח ותזיז את מנוע ה-Base שמאלה.
- האות P תשלח ותזיז את מנוע ה-Elbow קדימה.
- האות p תשלח ותזיז את מנוע ה-Elbow אחורה.

פונקציות חשובות לזרוע

```
1 int angleToPulse(int ang, String name) {
2     int pulse = map(ang, 0, 180, SERVOMIN, SERVOMAX);
3     Serial.print("Angle: ");
4     Serial.print(ang);
5     Serial.print(" pulse: ");
6     Serial.print(pulse);
7     Serial.print(" part: ");
8     Serial.println(name);
9     return pulse;
10 }
```

הפונקציה הזאת נועדה בשביל לא להשתמש ב-pulse ולהקשות עלינו ולעשות את זה בעזרת זווית, הספרייה של ה-סרוו אומרת שה-pulse MAX הוא 600 וה-pulse MIN הוא 100 לכן בעזרת פונקציית map נוכל להתאים לפי הזווית את ה-pulse המתאים.

```
1 void wristServoCW() {
2     current_wrist2_servo_pos = current_wrist2_servo_pos - wrist2_servo_pos_increment;
3     current_wrist2_servo_pos = constrain(current_wrist2_servo_pos, wrist2_servo_min, wrist2_servo_max);
4     pwm.setPWM(wrist2_pin, 0, angleToPulse(current_wrist2_servo_pos, "Wrist 2"));
5     delay(response_time_4);
6     Serial.print("Wrist 2 - Right: ");
7     Serial.println(current_wrist2_servo_pos);
8 }
```

הפונקציה מזיזה את המנוע של ה-wrist2 עם כיוון השעון כגודל ה-increment ודואגת שהוא יישאר בתווח מתאים בעזרת ה-constrain. היא שולחת דרך הפונקציה של ה-setPWM את ה-pulse המתאים למנוע על פי הפונקציה angleToPulse שהוסברה קודם לכן.

כל הפונקציות של הזזת המנוע נראות מהצורה הזאת רק שלכל אחד יש increment שונה וב-constrain יש ערכי max ו-min מתאימים לכל מנוע. ה-delay שונה בניהם גם.

סיכום הפרויקט/רפלקציה

לסיכום, הפרויקט שלי נועד להוות אלטרנטיבה זולה ושימושית לזרועות רובוטיות קיימות, אני מקווה שהצלחתי להבעיר את כוונותיי ביצירת הזרוע ושהבנתם את תהליך העבודה שלי, את המנגנונים שלי וסה"כ את הפרויקט כולו.

אני למדתי הרבה במהלך הפרויקט גם בפן התכנותי והחומרתי.

למדתי איך לבנות לתכנן פרויקט איך לשפר אלגוריתמים ואיך לחפש פתרונות לבעיות אלגוריתמיות והן חומרתיות בצורה הירה ויעילה ושיפרתי את הידע שלי בסה"כ בהכל.

שמחתי לעבוד על הפרויקט המיוחד הזה ושמחתי שלמדתי ממנו הרבה יותר ממה שציפיתי.

רשימת רכיבים לפרויקט:

1. Servo MG996R – 6 מנועים:



2. PCA9685 – מודול 1:



3. Arduino Uno – בקר 1:



4. HC05 – 2 מודולים:



5. MPU6050 – 2 חיישנים:



6. Flex sensor – 3/4 חיישנים:



7. Arduino Nano – בקר 1:



8. LM317 – מייצב 1:



ביבליוגרפיה

1. ה-Flex Sensor:
 - [/https://lastminuteengineers.com/flex-sensor-arduino-tutorial](https://lastminuteengineers.com/flex-sensor-arduino-tutorial)
2. ה-Servo MG996R:
 - https://www.electronicoscaldas.com/datasheet/MG996R_Tower-Pro.pdf
3. ה-PCA9685:
 - https://robojax.com/learn/arduino/?vid=robojax_PCA9685-V1
4. ה-MPU6050 וה-complementary filter:
 - https://www.youtube.com/watch?v=7VW_XVbtu9k
 - <https://www.nxp.com/docs/en/application-note/AN3461.pdf>
 - <https://stackoverflow.com/questions/55866808/attitude-estimation-from-accelerometer-and-gyroscope-of-an-imu>
 - <https://ahrs.readthedocs.io/en/latest/filters/complementary.html>
 - <https://github.com/MarkSherstan/MPU-6050-9250-I2C-CompFilter/blob/master/Arduino/main/main.ino>
5. מחלק מתח הסבר:
 - [/https://studymind.co.uk/notes/potential-dividers](https://studymind.co.uk/notes/potential-dividers)
6. רכיב התקשורת – HC05:
 - https://s3-sa-east-1.amazonaws.com/robocore-lojavirtual/709/HC-05_ATCommandSet.pdf
 - <https://www.youtube.com/watch?v=BXXAcFOTnBo>
7. הרכבת היד:
 - <https://www.youtube.com/watch?v=NqL9n3avBbY>
 - https://www.youtube.com/watch?v=hTZ2z_C9dSU
8. מייצב מתח:
 - https://www.ti.com/lit/ds/symlink/lm317.pdf?ts=1716559041492&ref_url=https%253A%252F%252Fwww.google.com%252F

תמונות של הפרויקט:



קוד פרויקט

קוד הכפפה

```
#include <Wire.h>
#include <Adafruit_MPU6050.h>
#include <Adafruit_Sensor.h>

#define MPU6050_ACCEL_FS_SEL 16384.0 // Sensitivity
scale factor for accelerometer (+/- 2g)
#define MPU6050_GYRO_FS_SEL 131.0 // Sensitivity
scale factor for gyroscope (+/- 250 deg/s)
#define NUM_SAMPLES 5000 // Number of
samples for calibration

const float VCC = 5; // voltage at Arduinio 5V
line
const float R_DIV = 10000.0; // resistor used to
create a voltage divider
const float flatResistance = 25000.0; // resistance
when flat
const float bendResistance = 62500.0; // resistance at
45 deg

Adafruit_MPU6050 mpu;
Adafruit_MPU6050 mpu2;

// Variables for first gyro calibration
float PGYROOFFSETX = 0.0;
float PGYROOFFSETY = 0.0;
float PGYROOFFSETZ = 0.0;

// Variables for second gyro calibration
float PGYROOFFSETX2 = 0.0;
```

```

float PGYROOFFSETY2 = 0.0;
float PGYROOFFSETZ2 = 0.0;

// Variables for angle calculation first MPU6050
float pitch1 = 0.0;
float roll1 = 0.0;
float yaw1 = 0.0;

// Variables for angle calculation second MPU6050
float pitch2 = 0.0;
float roll2 = 0.0;
float yaw2 = 0.0;

// Constants for complementary filter
float alpha = 0.98; // Weight for gyroscope data
unsigned long lastTime1 = 0;
unsigned long lastTime2 = 0;
float dt = 0.01; // Sampling interval (100 Hz)

//Create flex Sensors
int ADCpinkie = 0; //Pinkie flex
int ADCfinger = 0; //finger flex
int ADCthumb = 0; //thumb flex

float anglePinkie = 0.0;
float angleFinger = 0.0;
float angleThumb = 0.0;

float Vpinkie = 0.0;
float Vfinger = 0.0;
float Vthumb = 0.0;

```

```

float Rpinkie = 0.0;
float Rfinger = 0.0;
float Rthumb = 0.0;

int pinkie_Data = A3;
int finger_Data = A2;
int thumb_Data = A1;

int thumb_high = 11;
int thumb_low = 7;
int finger_high = 15;
int finger_low = 15;
int pinkie_high = 20;
int pinkie_low = 10;

//Stop Caliberating the Flex Sensor when complete
bool bool_caliberate = false;

//How often to send values to the Robotic Arm
int response_time = 100;

void setup() {
  pinMode(3, OUTPUT);
  Serial.begin(9600);
  while (!Serial) {
    delay(10);
  }

  if (!mpu.begin() || !mpu2.begin(0x69)) {
    Serial.println("Failed to find MPU6050 chip");
    while (1) {
      delay(10);
    }
  }
}

```

```

mpu.setAccelerometerRange(MPU6050_RANGE_2_G);
mpu.setGyroRange(MPU6050_RANGE_250_DEG);
mpu.setFilterBandwidth(MPU6050_BAND_21_HZ);

mpu2.setAccelerometerRange(MPU6050_RANGE_2_G);
mpu2.setGyroRange(MPU6050_RANGE_250_DEG);
mpu2.setFilterBandwidth(MPU6050_BAND_21_HZ);

// Perform gyro calibration - I don't use this now
// calibrateGyros();
delay(1000);
}

void loop() {
    digitalWrite(3, HIGH); //Use basic LED as visual
indicator if glove working

    //debug_flex(); //Debug Mode on/off

    //get values for first mpu having address of 0x68 -
default
    getmpu1filter();

    delay(10);

    //get values for second mpu having address of 0x69
    getmpu2filter();

    delay(10);

    send_mpu_data();

    get_flex_values();

```

```

    delay(20);

    send_flex_data();
}

void send_flex_data(){
    // Pinkie - (Elbow)
    if (anglePinkie < pinkie_high && anglePinkie >=
pinkie_low) {
        Serial.print("P");
        delay(response_time);

    }
    if (anglePinkie >= pinkie_high) {
        Serial.print("p");
        delay(response_time);
    }

    // thumb 1 - thumb (Base Rotation)
    if (angleThumb >= thumb_high) {
        Serial.print("T");
        delay(response_time);
    }

    if (angleThumb >= thumb_low && angleThumb <
thumb_high) {
        Serial.print("t");
        delay(response_time);
    }

    // finger 1 - Claw Bend/Open

```

```

    if (angleFinger >= finger_high) {
        Serial.print("f");
        delay(response_time);
    }

    if (angleFinger <= finger_low) {
        Serial.print("F");
        delay(response_time);
    }
    else {
        delay(5);
    }
}

void get_flex_values(){
    // read the values from Flex Sensors to Arduino
    ADCpinkie = analogRead(pinkie_Data);
    ADCfinger = analogRead(finger_Data);
    ADCthumb = analogRead(thumb_Data);

    Vpinkie = ADCpinkie * VCC / 1023.0;
    Vfinger = ADCfinger * VCC / 1023.0;
    Vthumb = ADCthumb * VCC / 1023.0;

    Rpinkie = R_DIV * (VCC / Vpinkie - 1.0);
    Rfinger = R_DIV * (VCC / Vfinger - 1.0);
    Rthumb = R_DIV * (VCC / Vthumb - 1.0);

    anglePinkie = map(Rpinkie, flatResistance,
bendResistance, 0, 45.0);
    angleFinger = map(Rfinger, flatResistance,
bendResistance, 0, 45.0);

```

```

    angleThumb = map(Rthumb, flatResistance,
bendResistance, 0, 45.0);
}

void debug_flex() {
    //Sends value as a serial monitor to port
    //Thumb
    Serial.print("Thumb: ");
    Serial.print(angleThumb);
    Serial.print("\t");

    //Thumb Params
    Serial.print("thumb High: ");
    Serial.print(thumb_high);
    Serial.print("\t");
    Serial.print("T Low: ");
    Serial.print(thumb_low);
    Serial.print("\t");

    //Finger
    Serial.print("finger: ");
    Serial.print(angleFinger);
    Serial.print("\t");

    //Finger Params
    Serial.print("finger High: ");
    Serial.print(finger_high);
    Serial.print("\t");
    Serial.print("finger Low: ");
    Serial.print(finger_low);
    Serial.print("\t");

    //Pinkie (Claw Further)
    Serial.print("Pinkie: ");

```



```

Serial.print(anglePinkie);
Serial.print("\t");

//Pinkie Params
Serial.print("Pinkie High: ");
Serial.print(pinkie_high);
Serial.print("\t");
Serial.print("Pinkie Low: ");
Serial.print(pinkie_low);
Serial.print("\t");
Serial.println();
}

void send_mpu_data(){
    //Move Left
    if ( (roll1 * 180 / M_PI) >= 40 && ((pitch1 * 180 /
M_PI) >= -25 && (pitch1 * 180 / M_PI) <= 25)) {
        Serial.print("L");
        delay(response_time);
    }

    //Move Right
    if ( (roll1 * 180 / M_PI) <= -80 && ((pitch1 * 180 /
M_PI) >= -25 && (pitch1 * 180 / M_PI) <= 25)) {
        Serial.print("R");
        delay(response_time);
    }

    //Claw Up
    if ( (pitch1 * 180 / M_PI) >= 45 ) {
        Serial.print("U"); // G
        delay(response_time);
    }
}

```

```

//Claw Down
if ( (pitch1 * 180 / M_PI) <= -45 ) {
    Serial.print("D"); // U
    delay(response_time);
}

//Move to me
if ( pitch2 * 180 / M_PI >= 45) { /// yaw1 * 180 /
M_PI >= -0.17) {
    Serial.print("C");
    delay(response_time);
}

//Move from me
if ( pitch2 * 180 / M_PI <= -45) { // || yaw1 * 180
/ M_PI <= -0.55
    Serial.print("c");
    delay(response_time);
}
}

void getmpu1filter(){
    // Read accelerometer and gyroscope data
    sensors_event_t accel, gyro, temp;
    mpu.getEvent(&accel, &gyro, &temp);

    // Apply gyro calibration offsets
    gyro.gyro.x -= PGYRO0FFSETX;
    gyro.gyro.y -= PGYRO0FFSETY;
    gyro.gyro.z -= PGYRO0FFSETZ;
}

```

```

    // Calculate angle from accelerometer data
    // pitch1 = atan2(accel.acceleration.y,
sqrt(pow(accel.acceleration.x, 2) +
pow(accel.acceleration.z, 2))); 2nd option
    // roll1 = atan2(-accel.acceleration.x,
accel.acceleration.z); 2nd option

    pitch1 = atan2(-accel.acceleration.x,
sqrt(pow(accel.acceleration.y, 2) +
pow(accel.acceleration.z, 2)));
    roll1 = atan2(accel.acceleration.y,
accel.acceleration.z);

    // Calculate change in time
    unsigned long currentTime = millis();
    float deltaTime = (currentTime - lastTime1) /
1000.0; // Convert to seconds
    lastTime1 = currentTime;

    // Calculate angle change from gyroscope data
    float gyro_pitch = gyro.gyro.x * (deltaTime /
MPU6050_GYRO_FS_SEL);
    float gyro_roll = gyro.gyro.y * (deltaTime /
MPU6050_GYRO_FS_SEL);
    float gyro_yaw = gyro.gyro.z * (deltaTime /
MPU6050_GYRO_FS_SEL);

    // Combine angles using complementary filter
    pitch1 = alpha * (pitch1 + gyro_pitch) + (1 - alpha)
* pitch1;
    roll1 = alpha * (roll1 + gyro_roll) + (1 - alpha) *
roll1;
    yaw1 += gyro_yaw;

```

```

/* Print filtered angles - debug
  Serial.print(250);
  Serial.print("Pitch Angle: ");
  Serial.print(",");
  Serial.print(pitch1 * 180 / M_PI); // Convert to
degrees
  Serial.print(" Roll Angle: ");
  Serial.print(",");
  Serial.print(roll1 * 180 / M_PI); // Convert to
degrees
  Serial.print(" Yaw Angle: ");
  Serial.print(",");
  Serial.print(yaw1 * 180 / M_PI); // Convert to
degrees
*/

// Add a small delay
delay(dt * 1000); // Convert to milliseconds
}

void getmpu2filter(){
  // Read accelerometer and gyroscope data
  sensors_event_t accel, gyro, temp;
  mpu2.getEvent(&accel, &gyro, &temp);

  // Apply gyro calibration offsets
  gyro.gyro.x -= PGYROOFFSETX2;
  gyro.gyro.y -= PGYROOFFSETY2;
  gyro.gyro.z -= PGYROOFFSETZ2;

  // Calculate angle from accelerometer data

```

```

    // pitch2 = atan2(accel.acceleration.y,
sqrt(pow(accel.acceleration.x, 2) +
pow(accel.acceleration.z, 2))); 2nd option
    // roll2 = atan2(-accel.acceleration.x,
accel.acceleration.z); 2nd option

    pitch2 = atan2(-accel.acceleration.x,
sqrt(pow(accel.acceleration.y, 2) +
pow(accel.acceleration.z, 2)));
    roll2 = atan2(accel.acceleration.y,
accel.acceleration.z);

    // Calculate change in time
    unsigned long currentTime = millis();
    float deltaTime = (currentTime - lastTime2) /
1000.0; // Convert to seconds
    lastTime2 = currentTime;

    // Calculate angle change from gyroscope data
    float gyro_pitch = gyro.gyro.x * (deltaTime /
MPU6050_GYRO_FS_SEL);
    float gyro_roll = gyro.gyro.y * (deltaTime /
MPU6050_GYRO_FS_SEL);
    float gyro_yaw = gyro.gyro.z * (deltaTime /
MPU6050_GYRO_FS_SEL);

    // Combine angles using complementary filter
    pitch2 = alpha * (pitch2 + gyro_pitch) + (1 - alpha)
* pitch2;
    roll2 = alpha * (roll2 + gyro_roll) + (1 - alpha) *
roll2;
    yaw2 += gyro_yaw;

    /* Print filtered angles - debug

```

```

    Serial.print(250);
    Serial.print("Pitch Angle: ");
    Serial.print(",");
    Serial.print(pitch2 * 180 / M_PI); // Convert to
degrees
    Serial.print(" Roll Angle: ");
    Serial.print(",");
    Serial.println(roll2 * 180 / M_PI); // Convert to
degrees
    Serial.print(" Yaw Angle: ");
    Serial.print(",");
    Serial.println(yaw2 * 180 / M_PI); // Convert to
degrees
*/

    // Add a small delay
    delay(dt * 1000); // Convert to milliseconds
}

void calibrateGyros() {
    float gyroOffsetX = 0.0, gyroOffsetY = 0.0,
    gyroOffsetZ = 0.0, gyroOffsetX2 = 0.0, gyroOffsetY2 =
0.0, gyroOffsetZ2 = 0.0;

    for (int i = 0; i < NUM_SAMPLES; i++) {
        sensors_event_t accel, gyro, temp, accel2, gyro2,
temp2;
        mpu.getEvent(&accel, &gyro, &temp);
        mpu2.getEvent(&accel2, &gyro2, &temp2);

        gyroOffsetX += gyro.gyro.x;
        gyroOffsetY += gyro.gyro.y;

```

```

gyroOffsetZ += gyro.gyro.z;
gyroOffsetX2 += gyro2.gyro.x;
gyroOffsetY2 += gyro2.gyro.y;
gyroOffsetZ2 += gyro2.gyro.z;

if(i%100 == 0) Serial.print(".");

delay(3); // Delay for stability
}

//Calculate average offset values
gyroOffsetX /= NUM_SAMPLES;
gyroOffsetY /= NUM_SAMPLES;
gyroOffsetZ /= NUM_SAMPLES;

gyroOffsetX2 /= NUM_SAMPLES;
gyroOffsetY2 /= NUM_SAMPLES;
gyroOffsetZ2 /= NUM_SAMPLES;

PGYRO0FFSETX = gyroOffsetX;
PGYRO0FFSETY = gyroOffsetY;
PGYRO0FFSETZ = gyroOffsetZ;

PGYRO0FFSETX2 = gyroOffsetX2;
PGYRO0FFSETY2 = gyroOffsetY2;
PGYRO0FFSETZ2 = gyroOffsetZ2;

// Print the calibration offsets
Serial.print("Gyro Offset X: ");
Serial.println(gyroOffsetX);
Serial.print("Gyro Offset Y: ");
Serial.println(gyroOffsetY);
Serial.print("Gyro Offset Z: ");
Serial.println(gyroOffsetZ);

```

```
    Serial.print("Gyro-2 Offset X: ");  
Serial.println(gyroOffsetX2);  
    Serial.print("Gyro-2 Offset Y: ");  
Serial.println(gyroOffsetY2);  
    Serial.print("Gyro-2 Offset Z: ");  
Serial.println(gyroOffsetZ2);  
}
```

קוד הזרוע

```
#include <Wire.h>  
#include <Adafruit_PWMServoDriver.h>  
#include <SoftwareSerial.h>  
  
#define RX_PIN 10  
#define TX_PIN 11  
  
SoftwareSerial BluetoothSerial(RX_PIN, TX_PIN); // RX,  
TX for Bluetooth module  
  
#define SERVOMIN 100 // This is the 'minimum' pulse  
length count (out of 4096)  
#define SERVOMAX 600 // This is the 'maximum' pulse  
length count (out of 4096)  
  
Adafruit_PWMServoDriver pwm =  
Adafruit_PWMServoDriver();  
  
const int base_pin = 8;  
const int shoulder_pin = 15;  
const int elbow_pin = 4;  
const int wrist1_pin = 7;  
const int wrist2_pin = 0;
```



```
const int gripper_pin = 12;

//initial parking position of the motor
const int base_servo_pos = 90;
const int shoulder_servo_pos = 45;
const int elbow_servo_pos = 45;
const int wrist1_servo_pos = 180;
const int wrist2_servo_pos = 90;
const int gripper_servo_pos = 90;

//Keep track of the current value of the motor
positions
int current_base_servo_pos = base_servo_pos;
int current_shoulder_servo_pos = shoulder_servo_pos;
int current_elbow_servo_pos = elbow_servo_pos;
int current_wrist1_servo_pos = wrist1_servo_pos;
int current_wrist2_servo_pos = wrist2_servo_pos;
int current_gripper_servo_pos = gripper_servo_pos;

//Degree of robot servo sensitivity - Intervals
int base_servo_pos_increment = 5;
int shoulder_servo_pos_increment = 5;
int elbow_servo_pos_increment = 5;
int wrist1_servo_pos_increment = 5;
int wrist2_servo_pos_increment = 5;

int gripper_servo_min = 80;
int gripper_servo_max = 0;

int wrist2_servo_min = 0;
int wrist2_servo_max = 180;

int wrist1_servo_min = 0;
int wrist1_servo_max = 180;
```

```

int elbow_servo_min = 0;
int elbow_servo_max = 180;

int shoulder_servo_min = 0;
int shoulder_servo_max = 180;

int base_servo_min = 0;
int base_servo_max = 180;

char state = 0; // Changes value from ASCII to char
int response_time_fast = 5;
int response_time_4 = 2;
int loop_check = 0;
int response_time = 100; //20
int action_delay = 600;

//Position of motor for example demos
int Pos;

void setup() {
    Serial.begin(9600);
    BluetoothSerial.begin(9600);
    pwm.begin();
    pwm.setPWMFreq(60); // Analog servos run at ~60 Hz
updates
    delay(3000);
    // wakeUp();
    start_pos();
}

```

```
void loop() {
    if (BluetoothSerial.available() > 0) { // Checks
        whether data is coming from the serial port

        state = BluetoothSerial.read(); // Reads the data
        from the serial port
        Serial.print(state); // Prints out the value sent

        //Move (Base Rotation) Stepper Motor Left
        if (state == 't') {
            baseRotateLeft();
            delay(response_time_fast);
        }

        //Move (Base Rotation) Stepper Motor Right
        if (state == 'T') {
            baseRotateRight();
            delay(response_time_fast);
        }

        //Move Shoulder Down
        if (state == 'c') {
            shoulderServoForward();
            delay(response_time_fast);
        }

        //Move Shoulder Up
        if (state == 'C') {
            shoulderServoBackward();
            delay(response_time_fast);
        }
    }
}
```

```
//Move Elbow Down
if (state == 'p') {
    elbowServoForward();
    delay(response_time_fast);
}

//Move Elbow Up
if (state == 'P') {
    elbowServoBackward();
    delay(response_time_fast);
}

//Move Wrist 1 UP
if (state == 'U') {
    wristServoUp();
    delay(response_time_fast);
}

//Move Wrist 1 Down
if (state == 'D') {
    wristServoDown();
    delay(response_time_fast);
}

//Move Wrist 2 Clockwise
if (state == 'R') {
    wristServoCW();
    delay(response_time_fast);
}
```

```

//Move Wrist 2 Counter-CW
if (state == 'L') {
    wristServoCCW();
    delay(response_time_fast);
}

//Open Claw Grip
if (state == 'F') {
    gripperServoOpen();
    delay(response_time_fast);
}

//Close Claw Grip
if (state == 'f') {
    gripperServoClose();
    delay(response_time_fast);
}

}

}

//Boiler plate function - These functions move the
servo motors in a specific direction for a duration.

void gripperServoClose() {
    pwm.setPWM(gripper_pin, 0,
angleToPulse(gripper_servo_min, "gripper"));
    delay(response_time_fast); //Delay the time takee to
turn the servo by the given increment
    Serial.println(gripper_servo_min);
}

```

```

void gripperServoOpen() {
    pwm.setPWM(gripper_pin, 0,
angleToPulse(gripper_servo_max, "gripper"));
    delay(response_time_fast);
    Serial.println(gripper_servo_max);
}

void wristServoCW() {
    current_wrist2_servo_pos = current_wrist2_servo_pos
- wrist2_servo_pos_increment;
    current_wrist2_servo_pos =
constrain(current_wrist2_servo_pos, wrist2_servo_min,
wrist2_servo_max);
    pwm.setPWM(wrist2_pin, 0,
angleToPulse(current_wrist2_servo_pos, "Wrist 2"));
    delay(response_time_4);
    Serial.print("Wrist 2 - Right: ");
    Serial.println(current_wrist2_servo_pos);
}

void wristServoCCW() {
    current_wrist2_servo_pos = current_wrist2_servo_pos
+ wrist2_servo_pos_increment;
    current_wrist2_servo_pos =
constrain(current_wrist2_servo_pos, wrist2_servo_min,
wrist2_servo_max);
    pwm.setPWM(wrist2_pin, 0,
angleToPulse(current_wrist2_servo_pos, "Wrist 2"));
    delay(response_time_4);
    Serial.print("Wrist 2 - Left: ");
    Serial.println(current_wrist2_servo_pos);
}

void wristServoDown() {

```

```

    current_wrist1_servo_pos = current_wrist1_servo_pos
+ wrist1_servo_pos_increment;
    current_wrist1_servo_pos =
constrain(current_wrist1_servo_pos, wrist1_servo_min,
wrist1_servo_max);
    pwm.setPWM(wrist1_pin, 0,
angleToPulse(current_wrist1_servo_pos, "Wrist 1"));
    delay(response_time);
    Serial.print("Wrist 1 - Down: ");
    Serial.println(current_wrist1_servo_pos);
}

void wristServoUp() {
    current_wrist1_servo_pos = current_wrist1_servo_pos
- wrist1_servo_pos_increment;
    current_wrist1_servo_pos =
constrain(current_wrist1_servo_pos, wrist1_servo_min,
wrist1_servo_max);
    pwm.setPWM(wrist1_pin, 0,
angleToPulse(current_wrist1_servo_pos, "Wrist 1"));
    delay(response_time);
    Serial.print("Wrist 1 - Up: ");
    Serial.println(current_wrist1_servo_pos);
}

void elbowServoForward() {
    current_elbow_servo_pos = current_elbow_servo_pos -
elbow_servo_pos_increment;
    current_elbow_servo_pos =
constrain(current_elbow_servo_pos, elbow_servo_min,
elbow_servo_max);
    pwm.setPWM(elbow_pin, 0,
angleToPulse(current_elbow_servo_pos, "Elbow"));
    delay(response_time);

```

```

    Serial.print("Elbow - Down: ");
    Serial.println(current_elbow_servo_pos);
}

void elbowServoBackward() {
    current_elbow_servo_pos = current_elbow_servo_pos +
    elbow_servo_pos_increment;
    current_elbow_servo_pos =
    constrain(current_elbow_servo_pos, elbow_servo_min,
    elbow_servo_max);
    pwm.setPWM(elbow_pin, 0,
    angleToPulse(current_elbow_servo_pos, "Elbow"));
    delay(response_time);
    Serial.print("Elbow - Up: ");
    Serial.println(current_elbow_servo_pos);
}

void shoulderServoForward() {
    current_shoulder_servo_pos =
    current_shoulder_servo_pos -
    shoulder_servo_pos_increment;
    current_shoulder_servo_pos =
    constrain(current_shoulder_servo_pos,
    shoulder_servo_min, shoulder_servo_max);
    pwm.setPWM(shoulder_pin, 0,
    angleToPulse(current_shoulder_servo_pos, "Shoulder"));
    delay(response_time);
    Serial.print("Shoulder - Stretch: ");
    Serial.println(current_shoulder_servo_pos);
}

void shoulderServoBackward() {

```



```

    current_shoulder_servo_pos =
current_shoulder_servo_pos +
shoulder_servo_pos_increment;
    current_shoulder_servo_pos =
constrain(current_shoulder_servo_pos,
shoulder_servo_min, shoulder_servo_max);
    pwm.setPWM(shoulder_pin, 0,
angleToPulse(current_shoulder_servo_pos, "Shoulder"));
    delay(response_time);
    Serial.print("Shoulder - Bend: ");
    Serial.println(current_shoulder_servo_pos);
}

void baseRotateLeft() {
    current_base_servo_pos = current_base_servo_pos -
base_servo_pos_increment;
    current_base_servo_pos =
constrain(current_base_servo_pos, base_servo_min,
base_servo_max);
    pwm.setPWM(base_pin, 0,
angleToPulse(current_base_servo_pos, "Base"));
    delay(response_time);
    Serial.print("Base - Left: ");
    Serial.println(current_base_servo_pos);
}

void baseRotateRight() {
    current_base_servo_pos = current_base_servo_pos +
base_servo_pos_increment;
    current_base_servo_pos =
constrain(current_base_servo_pos, base_servo_min,
base_servo_max);
    pwm.setPWM(base_pin, 0,
angleToPulse(current_base_servo_pos, "Base"));

```

```

    delay(response_time);
    Serial.print("Base - Right: ");
    Serial.println(current_base_servo_pos);
}

void wakeUp() {

    //Pre-Program Function - Wake Up Robot on Start

    // pwm.setPWM(base_pin, 0, 0);
    pwm.setPWM(shoulder_pin, 0, angleToPulse(0,
"shoulder"));
    pwm.setPWM(elbow_pin, 0, angleToPulse(0, "Elbow"));
    // pwm.setPWM(wrist1_pin, 0, 47);
    // pwm.setPWM(wrist2_pin, 0, 63);
    // pwm.setPWM(gripper_pin, 0, 63);

    if (loop_check == 0) {
        // //base move full
        for (Pos = 0; Pos < 180; Pos++)
        {

            pwm.setPWM(base_pin, 0, angleToPulse(Pos,
"base"));
            delay(response_time);
        }

        //base move center cw
        for (Pos = 180; Pos > 90; Pos--)
        {

            pwm.setPWM(base_pin, 0, angleToPulse(Pos,
"base"));
            delay(response_time);
        }
    }
}

```

```

    }

    // //base move center ccw
    // for (Pos = 0; Pos <= 90; Pos++)
    // {
    //     pwm.setPWM(base_pin, 0, angleToPulse(Pos,
"base"));
    //     delay(response_time);
    // }

    //Shoulder Raise
    for (Pos = 0; Pos < 45; Pos++)
    {

        pwm.setPWM(shoulder_pin, 0, angleToPulse(Pos,
"Shoulder"));
        delay(response_time);
    }

    // //Move Elbow Backwards
    for (Pos = 0; Pos < 45; Pos++)
    {

        pwm.setPWM(elbow_pin, 0, angleToPulse(Pos,
"Elbow"));

        delay(response_time);
    }

    //Move Wrist 1 Forward
    for (Pos = 180; Pos > 150; Pos--)
    {
        pwm.setPWM(wrist1_pin, 0, angleToPulse(Pos,
"Wrist 1"));
    }

```

```

    delay(response_time_fast);
}

//Move Wrist 2 Backwards
for (Pos = 90; Pos > 0; Pos--)
{
    if(Pos == 45){
        pwm.setPWM(gripper_pin, 0, angleToPulse(0,
"gripper"));
        delay(response_time_fast+95);
        pwm.setPWM(gripper_pin, 0, angleToPulse(80,
"gripper"));
        delay(response_time_fast+95);
    }
    pwm.setPWM(wrist2_pin, 0, angleToPulse(Pos,
"Wrist 2"));
    delay(response_time_fast);
}

//Move Wrist 2 Backwards
for (Pos = 0; Pos < 180; Pos++)
{
    if(Pos == 45){
        pwm.setPWM(gripper_pin, 0, angleToPulse(0,
"gripper"));
        delay(response_time_fast+55);
        pwm.setPWM(gripper_pin, 0, angleToPulse(80,
"gripper"));
        delay(response_time_fast+55);
    }
    pwm.setPWM(wrist2_pin, 0, angleToPulse(Pos,
"Wrist 2"));
    delay(response_time_fast);
}

```

```

    //Move Wrist 2 Center
    for (Pos = 180; Pos > 90; Pos--)
    {
        pwm.setPWM(wrist2_pin, 0, angleToPulse(Pos,
"Wrist 2"));
        delay(response_time_fast);
    }

    //Move Wrist 1 Forward
    for (Pos = 150; Pos < 180; Pos++)
    {
        pwm.setPWM(wrist1_pin, 0, angleToPulse(Pos,
"Wrist 1"));
        delay(response_time_fast);
    }

    //Move Elbow Forward
    for (Pos = 45; Pos > 0; Pos--)
    {
        pwm.setPWM(elbow_pin, 0, angleToPulse(Pos,
"Elbow"));
        delay(response_time_fast);
    }

    loop_check = 0;

}

}

void start_pos(){
    pwm.setPWM(shoulder_pin, 0, angleToPulse(0,
"shoulder"));
    pwm.setPWM(elbow_pin, 0, angleToPulse(0, "Elbow"));

```

```

    pwm.setPWM(wrist1_pin, 0, angleToPulse(180, "Wrist
1"));

    if (loop_check == 0) {
        // //base move full
        for (Pos = 0; Pos <= 90; Pos++)
        {

            pwm.setPWM(base_pin, 0, angleToPulse(Pos,
"base"));
            delay(response_time);
        }

        //Shoulder Raise
        for (Pos = 0; Pos <= 45; Pos++)
        {

            pwm.setPWM(shoulder_pin, 0, angleToPulse(Pos,
"Shoulder"));
            delay(response_time);
        }

        //Move Wrist 2 Backwards
        for (Pos = 90; Pos >= 0; Pos--)
        {
            if(Pos == 45){
                pwm.setPWM(gripper_pin, 0, angleToPulse(0,
"grripper"));
                delay(response_time_fast+55);
                pwm.setPWM(gripper_pin, 0, angleToPulse(90,
"grripper"));
                delay(response_time_fast+55);
            }
        }
    }

```

```

        pwm.setPWM(wrist2_pin, 0, angleToPulse(Pos,
"Wrist 2"));
        delay(response_time_fast);
    }

    //Move Wrist 2 Backwards
    for (Pos = 0; Pos <= 180; Pos++)
    {
        if(Pos == 135){
            pwm.setPWM(gripper_pin, 0, angleToPulse(0,
"gripper"));
            delay(response_time_fast+55);
            pwm.setPWM(gripper_pin, 0, angleToPulse(90,
"gripper"));
            delay(response_time_fast+55);
        }
        pwm.setPWM(wrist2_pin, 0, angleToPulse(Pos,
"Wrist 2"));
        delay(response_time_fast);
    }

    //Move Wrist 2 Center
    for (Pos = 180; Pos >= 90; Pos--)
    {
        pwm.setPWM(wrist2_pin, 0, angleToPulse(Pos,
"Wrist 2"));
        delay(response_time_fast);
    }

    loop_check = 0;
}

pwm.setPWM(gripper_pin, 0, angleToPulse(0,
"gripper"));

```

```

}

int angleToPulse(int ang, String name) {
    int pulse = map(ang, 0, 180, SERVOMIN, SERVOMAX);
    Serial.print("Angle: ");
    Serial.print(ang);
    Serial.print(" pulse: ");
    Serial.print(pulse);
    Serial.print(" part: ");
    Serial.println(name);
    return pulse;
}

    }

    loop_check = 0;
}

}

int angleToPulse(int ang, String name) {
    int pulse = map(ang, 0, 180, SERVOMIN, SERVOMAX);
    Serial.print("Angle: ");
    Serial.print(ang);
    Serial.print(" pulse: ");
    Serial.print(pulse);
    Serial.print(" part: ");
    Serial.println(name);
    return pulse;
}

```